

# LESSON 16

## Part A and B

Basic datasets management,  
Coding in Matlab and Colab,  
One-hot encoding





# Outline

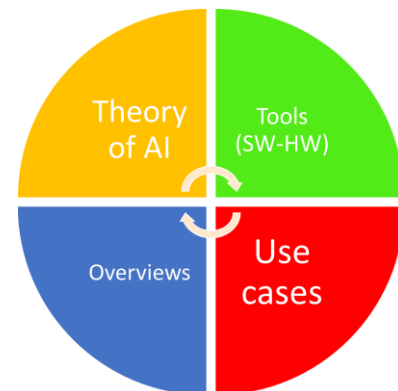
- Accuracy and confidence assessment for classifiers and regressors
  - $R^2$ , MSE, RMSE, confusion matrix, ROC, sensitivity analysis
- Basic datasets management
  - Coding in Matlab and Colab
  - Using reference dataset The IRIS dataset
- Hard encoding
- Loading and basic dataset management
  - A Matlab example
  - A Colab example
- Exploratory Data Analysis (EDA)



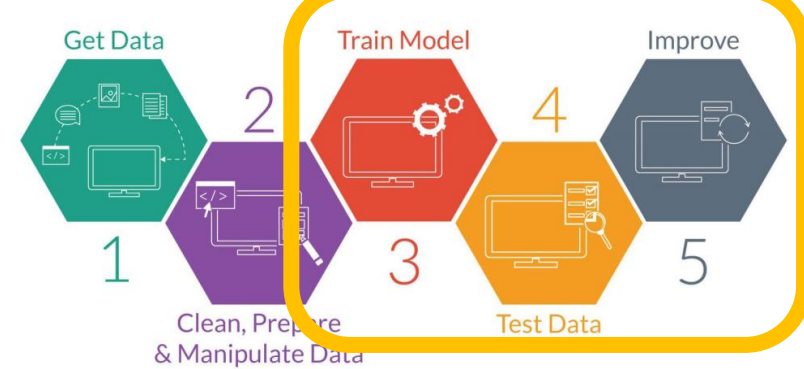
# THEORY

## Accuracy assessment

For classifiers and Regressors. Confidence



# Accuracy assessment



- In which step is used?
  - #3: By the learning method (optimization of the parameters) → Train Dataset
  - #4: By you → Test Data (generalization assessment)
  - #5: By you → Improving the whole systems (trying new models or learning methods etc.)
- General expression:  $Y = \text{function}(X, \text{params})$
- Different metrics
  - Regressors:  $Y \in R$  (Real numbers)
  - Classifiers:  $Y \in Z$  (Integers numbers)

# Regressors: errors assessment

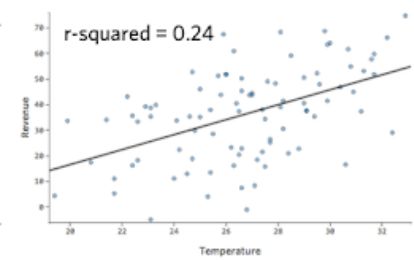
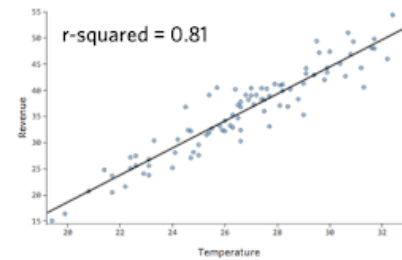


amazon

- Image you are creating for Amazon a regressor (predictor) based on the user data (age, city, level of instructions, time of the day, previous order amount, days by last order) will predict the **amount of euro of the next order (within a week)**
- `[euros_next_order] = function( [age, city, level_of_instructions, time_of_the_day, previous_order_amount, days_by_last_order])`
- How can you **prove that your regressor is good?**
- Based on **what features of merit** can you improve your regressor?

# Regressors

## R-squared (R2)



- $R^2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2} = 1 - \frac{\text{Residual Sum of Squares}}{\text{Total Sum of Squares}}$ 
  - Unexplained variation (w.r.t the output of the model)
  - Total variation (w.r.t the mean)
- Represents the proportion of variation in the outcome that is explained by the predictor variables.
- The Higher the R-squared, the better the model
- Range:  $[-\infty, 1]$  (for ad-hoc dataset or bad models) Usual range  $[0, 1]$
- Units: Adimensional

# Regressors:

## Root Mean Squared Error (RMSE)

- It measures the average error performed by the model in predicting the outcome for an observation.

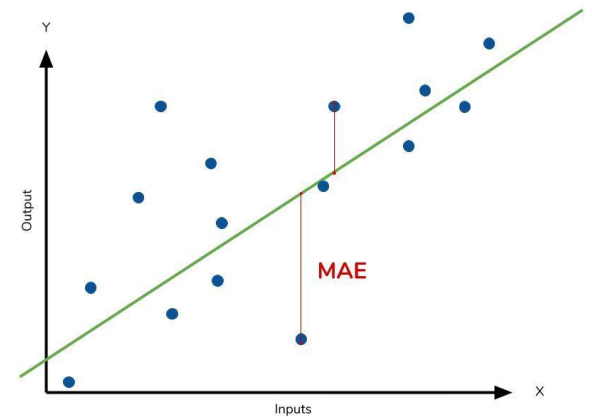
$$MSE = \sum_{i=1}^n \frac{(\hat{y}_i - y_i)^2}{n}$$

Sensitive to outliers!

$$RMSE = \sqrt{MSE}$$

- The lower the MSE/RMSE, the better the model  
**Range:**  $[0, +\infty]$ .  
**Units MSE:** squared unit of  $y$  (e.g. EUR<sup>2</sup>)  
**Units RMSE:** same unit of  $y$  (e.g. EUR)

# Regressors: Mean Absolute Error (MAE)



- MAE = mean(abs(observeds - predicted)).

$$\text{MAE} = \frac{1}{n} \sum_{j=1}^n |y_j - \hat{y}_j|$$

- MAE is less sensitive to outliers compared to MSE.
- The lower the MAE, the better the model  
**Range:**  $[0, +\infty]$ .  
**Units:** same as y variable (e.g. EUR)

# Classifiers: errors assessment

- Imagine you are creating for a hospital a classifier capable to early diagnose the pre-diabetes based on patient data and blood tests (sex, age, weight, glycated hemoglobin, ...)

[diabetes\_presence] = function( [sex, age, GL, ... ] )

- How can you prove that your classifier is good?
- Based on what features of merit can you improve your classifier? A typical one is Accuracy.

# Classifiers: errors assessments

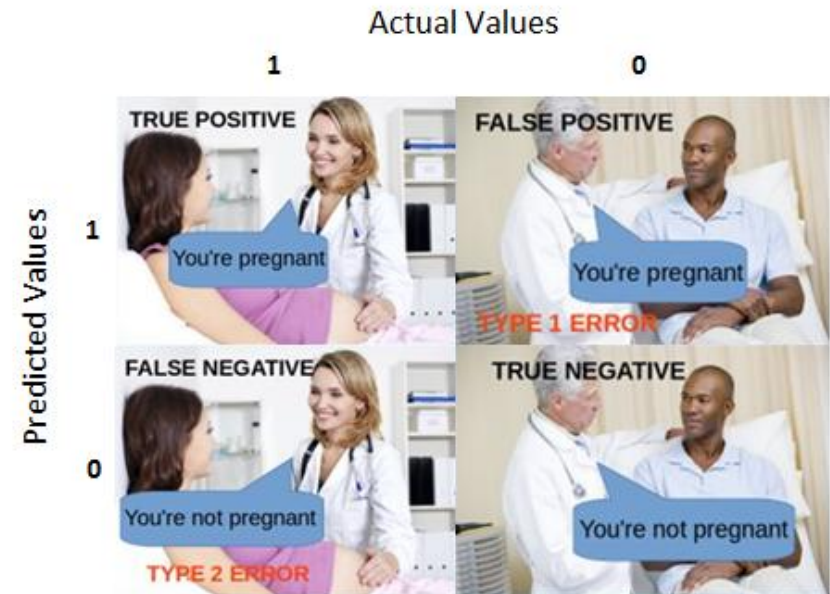
- Accuracy (%) is measured as the ratio (percentage) of predicted results that match the expected results.
  - If there are 1000 results, and 850 predicted results match the expected results, then the accuracy is 0.85 or 85%.
  - $\text{Accuracy} = 1 - \text{error}$  (sometime  $\text{accuracy} = \text{error}$ )

# Some good reference for Accuracy

- If the training and test data are skewed towards one class, then the model will predict everything as being that class.
  - Random CLASSIFIER:  
out = rand() // what is the accuracy? Sometimes 50%
  - “not-so-stupid” CLASSIFIER  
out = most\_present\_class (dataset\_set)
    - Ex. In Titanic training data, 68% of people died. If one trained a model to predict everybody died, it would be 68% accurate.
- Saying 1% error accuracy can be very good, or very bad!!

# Confusion Matrix

## 4 Quadrant Measurement



Number correctly predicted  
not as the class (e.g., not a dog)

Number incorrectly predicted  
as the class (e.g. dog), when it is  
not that class.

|                | Predicted (False)   | Predicted (True)    |
|----------------|---------------------|---------------------|
| Actual (False) | True Negative (TN)  | False Positive (FP) |
| Actual (True)  | False Negative (FN) | True Positive (TP)  |

Number incorrectly predicted  
as not the class (e.g. not a dog)

Number correctly predicted  
as the class (e.g., dog)

# Confusion Matrix

|                | Predicted (False)   | Predicted (True)    |
|----------------|---------------------|---------------------|
| Actual (False) | True Negative (TN)  | False Positive (FP) |
| Actual (True)  | False Negative (FN) | True Positive (TP)  |

## Example of measurements

- Accuracy =  $( TP + TN ) / N$
- Misclassification =  $( FP + FN ) / N$
- Precision =  $TP / ( TP + FP )$
- Sensitivity, recall, hit rate, or true positive rate (TPR)  
=  $TP / ( TP + FN )$
- Specificity, selectivity or true negative rate (TNR)  
=  $TN / ( TN + FP )$

**IMPORTANT!!!**

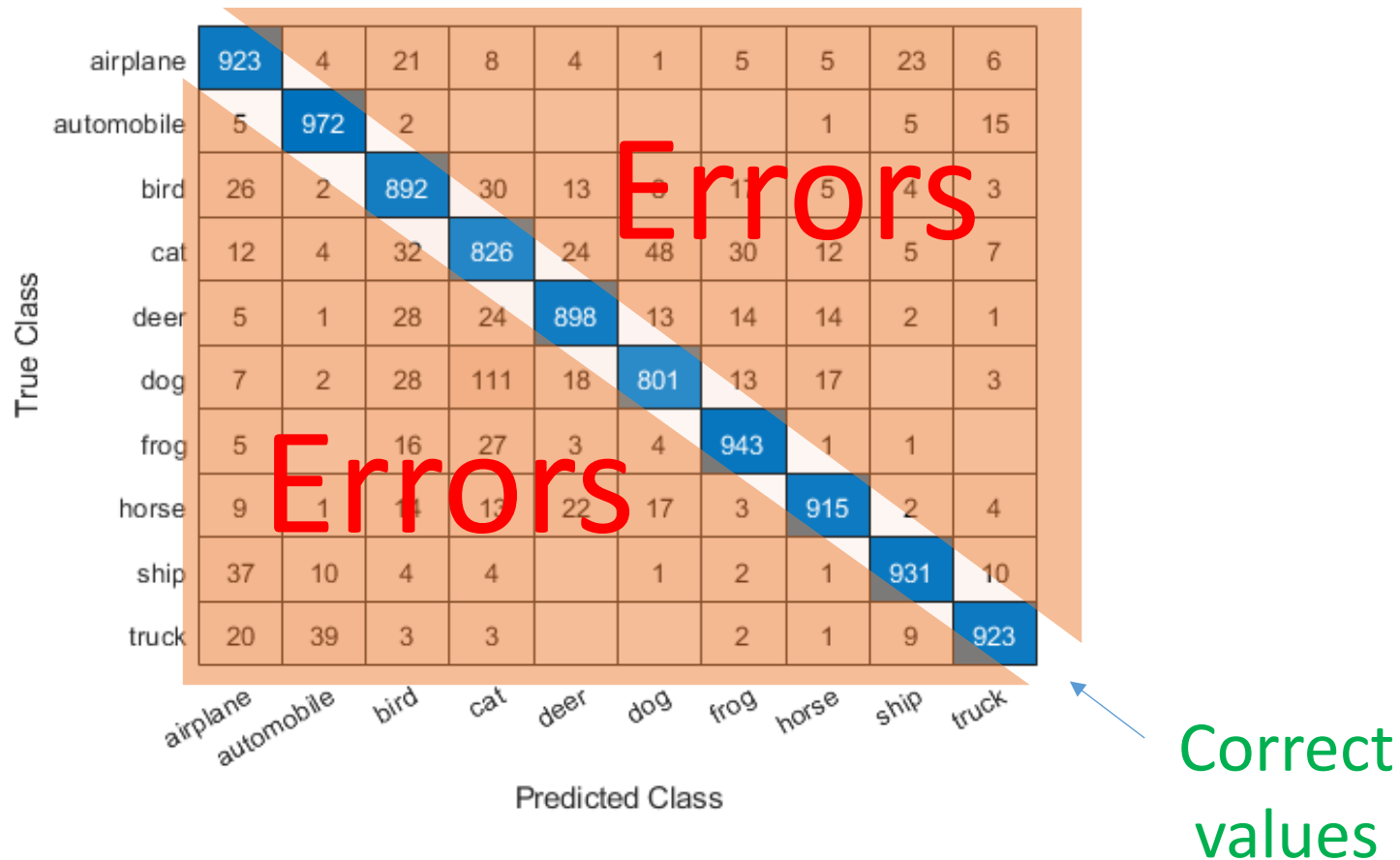
The confusion matrix gives more info about the errors than just the accuracy

# Confusion matrix (multiclass)

|            |            |                 |            |      |     |      |     |      |       |      |       |
|------------|------------|-----------------|------------|------|-----|------|-----|------|-------|------|-------|
| True Class | airplane   | 923             | 4          | 21   | 8   | 4    | 1   | 5    | 5     | 23   | 6     |
|            | automobile | 5               | 972        | 2    |     |      |     |      | 1     | 5    | 15    |
|            | bird       | 26              | 2          | 892  | 30  | 13   | 8   | 17   | 5     | 4    | 3     |
|            | cat        | 12              | 4          | 32   | 826 | 24   | 48  | 30   | 12    | 5    | 7     |
|            | deer       | 5               | 1          | 28   | 24  | 898  | 13  | 14   | 14    | 2    | 1     |
|            | dog        | 7               | 2          | 28   | 111 | 18   | 801 | 13   | 17    |      | 3     |
|            | frog       | 5               |            | 16   | 27  | 3    | 4   | 943  | 1     | 1    |       |
|            | horse      | 9               | 1          | 14   | 13  | 22   | 17  | 3    | 915   | 2    | 4     |
|            | ship       | 37              | 10         | 4    | 4   |      | 1   | 2    | 1     | 931  | 10    |
|            | truck      | 20              | 39         | 3    | 3   |      |     | 2    | 1     | 9    | 923   |
|            |            | Predicted Class |            |      |     |      |     |      |       |      |       |
|            |            | airplane        | automobile | bird | cat | deer | dog | frog | horse | ship | truck |

Cifar10Labels.mat

# Confusion matrix (multiclass) how to read it



# Confusion matrix (multiclass) how to read it

26 real birds  
have been confused  
with airplanes

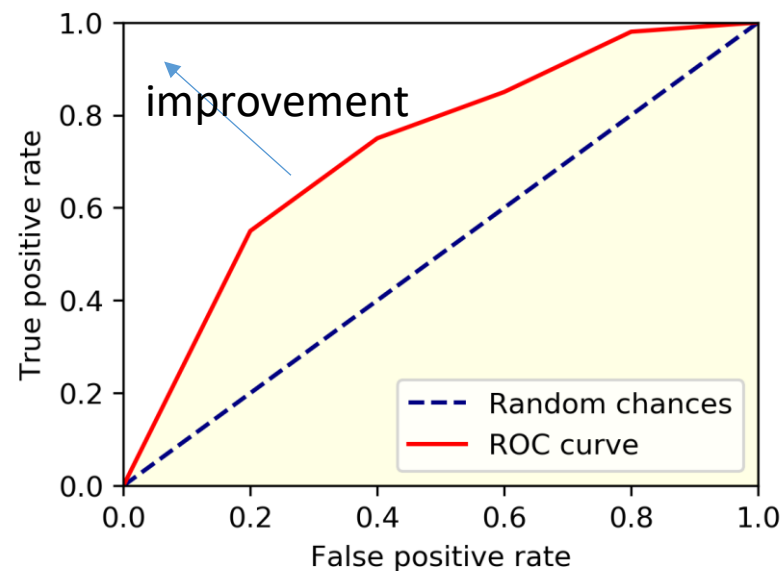
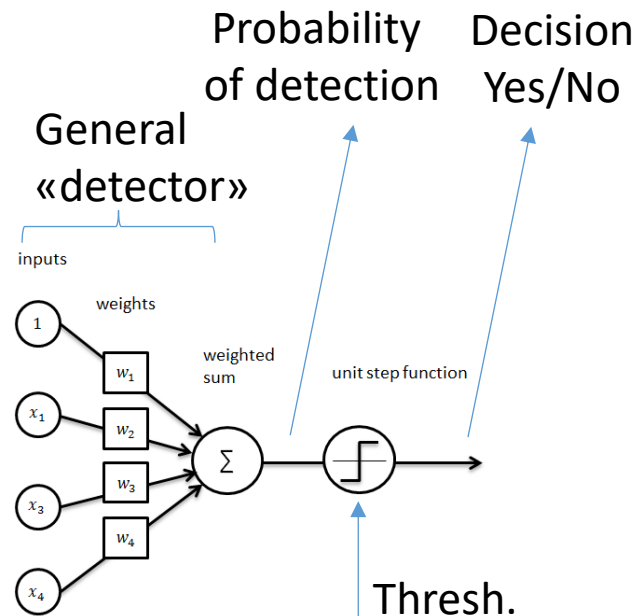
111 real dogs  
have been confused  
with cats

| True Class      | airplane | automobile | bird | cat | deer | dog | frog | horse | sheep | truck |
|-----------------|----------|------------|------|-----|------|-----|------|-------|-------|-------|
|                 | 923      | 4          | 21   | 8   | 4    | 1   | 5    | 5     | 23    | 6     |
|                 | 5        | 972        | 2    |     |      |     |      | 1     | 5     | 15    |
|                 | 26       | 2          | 892  | 30  | 13   | 8   | 17   | 5     | 4     | 3     |
|                 | 12       | 4          | 32   | 826 | 24   | 48  | 30   | 12    | 5     | 7     |
|                 | 5        | 1          | 28   | 24  | 898  | 13  | 14   | 14    | 2     | 1     |
|                 | 7        | 2          | 28   | 111 | 18   | 801 | 13   | 17    |       | 3     |
|                 | 5        |            | 16   | 27  |      | 4   | 943  | 1     | 1     |       |
|                 | 9        | 1          | 14   | 13  | 22   |     | 3    | 915   | 2     | 4     |
|                 | 37       | 10         | 4    | 4   |      | 1   |      | 1     | 931   | 10    |
| Predicted Class | airplane | automobile | bird | cat | deer | dog | frog | horse | sheep | truck |
|                 | 20       | 39         | 3    | 3   |      |     | 2    |       | 9     | 923   |

Need more  
examples?

# Receiver operating characteristic (ROC)

- Not just a single number but a graphical plot of the errors of a binary classifier system as its discrimination threshold is varied.
- The ROC curve is created by plotting the true positive rate (TPR) against the false positive rate (FPR) at various threshold settings.





# Different New/Different Approach to Training

From Data-Driven Validation to Representation-Driven  
Adaptation

# The ML Pipeline Evolution

- **Traditional Pipeline**

Data Collection → Feature Engineering → Training → Cross-Validation → Deployment

- **New (Additional) Pipeline**

Data analysis of supervised and unsupervised data → Pre-trained Foundation Models → Semantic Grounding → Zero/Few-Shot Adaptation

→ Human-in-the-Loop Validation

The validation bottleneck shifts from **data availability** to **representation quality and semantic grounding**.

# Foundation Models in IS4

## What are Foundation Models?

- Large-scale models **pre-trained on massive and diverse datasets**
- Learn **general-purpose representations** transferable across tasks
- Adaptable via prompting, fine-tuning, or **zero/few-shot learning**

## Why they matter in IS4

- Provide strong **semantic understanding** of code, logs, and text
- Enable **reasoning on unseen threats** without task-specific training
- Serve as the backbone for **Zero-Shot and Few-Shot industrial systems (design and train, not for realtime deployment)**

Foundation Models shift ML from task-specific training to knowledge-driven adaptation.

# Zero shot and Few-shot models

## What they are NOT

- Not specific models
- Not training algorithms
- Not architectures

## What they ARE

- **Inference and adaptation paradigms**
- Ways to **use a trained model** on new tasks or classes

| Zero-shot                                | One-shot                                                | Few-shot                                                              |
|------------------------------------------|---------------------------------------------------------|-----------------------------------------------------------------------|
| Translate<br>"Good morning"<br>to French | "Hello" →<br>"Bonjour"<br>Now translate:<br>"Goodbye" → | "Hello" → "Bonjour"<br>"Yes" → "Oui"<br>"No" → "Non"<br>"Thank you" → |

# Zero shot / Few-shot

## Zero-Shot Learning (ZSL)

- The model performs a task **without seeing any labeled examples**
- Uses instructions, descriptions, or semantic knowledge only

## Few-Shot Learning (FSL)

- The model adapts using **a very small number of labeled examples** (e.g., 1–10)
- Relies on strong representations learned during pre-training

Zero-shot and few-shot describe **how**  
a model is used,  
not **what kind of model** it is.

# Zero shot

## Model Behavior

- No examples provided
- Decision based only on semantic knowledge
- Pattern recognition (obfuscation, hidden execution, encoded command)

Your prompt

You are an industrial maintenance analyst.

Is this machine behavior anomalous?

"The motor shows increasing vibration after startup, temperature rises by 12 °C in 20 minutes, and alarm A17 appears intermittently."

Output of the model

Likely anomalous behavior.

The pattern suggests a possible mechanical or thermal issue, such as bearing wear, misalignment, or insufficient lubrication. A maintenance inspection is recommended.

23

# Few shot

## Model Behavior

- Learns the pattern from few labeled examples
- Adapts its decision to the new attack family
- Stronger consistency on similar samples

### Your prompt

This is a new maintenance classification task.

Example 1 (anomalous):

"Vibration increases after startup, temperature rises by 15 °C, and alarm A17 appears intermittently."  
Reason: possible bearing wear or lubrication issue.

Example 2 (normal):

"Temperature remains stable, vibration is within tolerance, and no alarm is reported during the operating cycle."  
Reason: normal operating condition.

Example 3 (anomalous):

"Current absorption increases under constant load, vibration is above threshold, and the motor produces abnormal noise."  
Reason: possible mechanical friction or misalignment.

Now analyze this case:

"The motor shows increasing vibration after startup, temperature rises by 12 °C in 20 minutes, and alarm A17 appears intermittently."

Output of the model

Likely anomalous maintenance case.

The symptoms are close to Example 1: increasing vibration, temperature drift, and intermittent alarm A17.

A possible cause is bearing wear, misalignment, or insufficient lubrication.

# Using generic LLM model (ChatGPT, Gemini, etc.)

- From feature extraction and classification to “speaking with the LLM”

## Input Prompt

You are an industrial maintenance analyst.

Analyze the following maintenance event and answer:

- 1) Is the machine behavior likely normal or anomalous?
- 2) If anomalous, describe the most likely failure pattern.
- 3) Assign a confidence level low / medium / high.
- 4) Suggest the first maintenance checks.

Maintenance event:

Machine ID: MTR-17  
Operating phase: startup  
Vibration: increasing after 5 minutes  
Temperature: +12 °C in 20 minutes  
Current absorption: slightly above baseline  
Alarm: A17 intermittent  
Operator note: abnormal noise from motor housing

What I like to have....

(not yet exactly possible today to have high accuracy)

## Output Model Response

1) Likely anomalous behavior.

2) The pattern is compatible with an early mechanical or thermal fault. The combination of increasing vibration, temperature drift, slightly higher current absorption, intermittent alarm A17, and abnormal noise may indicate bearing wear, misalignment, mechanical friction, or insufficient lubrication.

3) Confidence level: Medium.

The evidence is coherent with a maintenance warning pattern, but the model has no access to historical reports, machine-specific thresholds, or previous A17 cases.

4) Suggested first checks:

- inspect bearing condition
- verify lubrication status
- check shaft alignment
- compare vibration spectrum with previous baseline
- inspect alarm A17 history in the maintenance system

# Evaluating Accuracy of Foundation Models in IS4

## Why evaluation is non-trivial

- Predictions are often **semantic and explanatory**, not simple class labels
- Zero/Few-Shot models operate under **data scarcity and distribution shift**
- Classical cross-validation assumptions may not hold

# Evaluating Accuracy of Foundation Models in IS4

## Evaluation Strategies

- **Label-Based Metrics (when possible)**  
Accuracy, Precision, Recall, F1-score  
(used when ground truth labels are available, even if limited)
- **Scenario-Based Evaluation**  
Test on **unseen attack families** or **time-split data**  
(train on past threats, evaluate on newer ones)
- **Human-in-the-Loop Validation**  
Security analysts review predictions and explanations  
Measure agreement rate and false positives
- **Confidence and Consistency Analysis**  
Evaluate prediction confidence and stability  
“This activity is malicious (high confidence)”  
“This activity may be malicious (low confidence)”

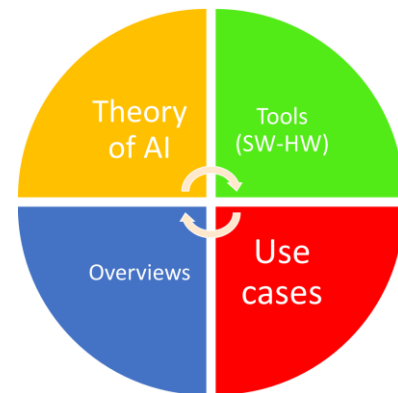
For foundation models, evaluation shifts from  
pure statistical accuracy to  
operational reliability under  
uncertainty.



# THEORY

## Other accuracy Assessment

Saliency maps, Injecton test, Sensitivity



# Try to modify the inputs



Input image



NN1

Dog = 0,95

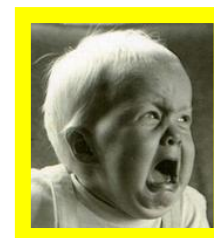


Input image



NN2

Dog = 0,89



# Check also with saliency maps



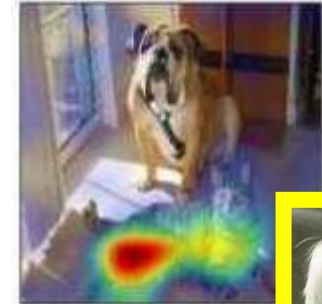
Input image



NN1

Dog = 0,95

Saliency map



The prediction is correct but,  
looking at the wrong pixels!!

Can you trust this model even it appears accurate?



Input image



NN2

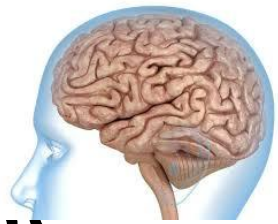
Dog = 0,92

Saliency map



# Adding a fair amount of noise

sensitivity analysis of a model



Input image



NN1

Dog = 0,95

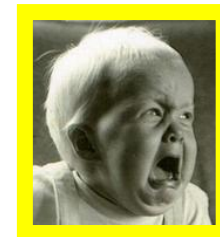


Input image



NN1

truck= 0,78



# Sensitivity Analysis

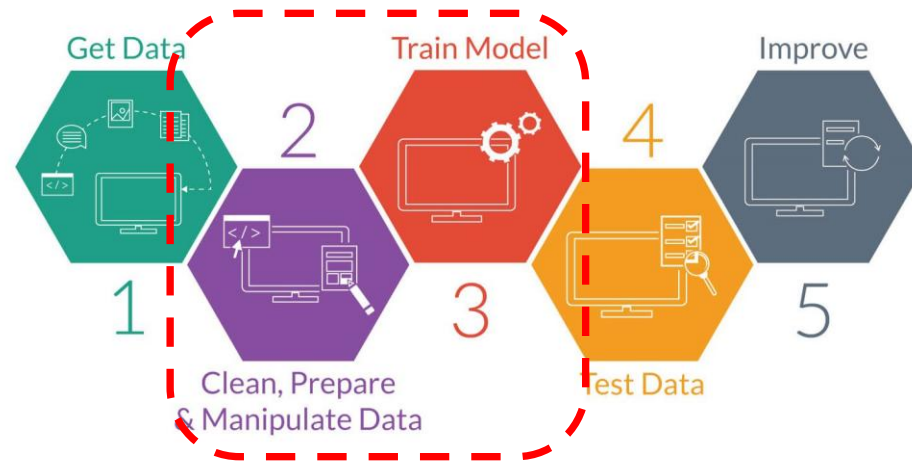
- Series of methods to quantify how the uncertainty in the output of a model is related to the uncertainty in its inputs or the internal representation
- Sensitivity analysis assesses how “sensitive” the model is to
  - fluctuations in the parameters
    - Example: I run the model on the very fast 8bit NPU and the outcomes are different than in my tensorflow@my PC!!
  - data on which it is built
    - I mix the data fra Train/Validation and the accuracy change a lot!

# Lesson outline

- Lesson = Theory + LABORATORY
- Using reference dataset
  - The IRIS dataset
- Hard encoding
- **Main points**
- **LABORATORY (in a separate file):**  
Loading and basic dataset management
  - A Matlab example
  - A Colab example



# Step 2 (and ready for Training..)



# GOLDEN RULE #1: use a simple well-known dataset to start

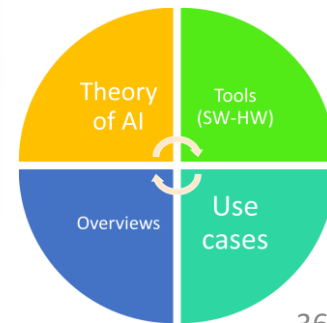
- To test your
  - knowledge,
  - data processing tools,
  - Models
  - learning methods
- Do not start with the actual dataset of your application!!
- Start with known and easy dataset with known outputs
  - Plots, accuracy, results are well documented, you can compare your outputs with the literature



# Toolboxes

## The Iris dataset

Start with something known  
every time you test a new tool.



# Example: iris flower

- Classify iris flowers into 3 classes:
  1. Setosa
  2. Versicolor
  3. Virginica
- 4 features:
  1. Sepal length in cm
  2. Sepal width in cm
  3. Petal length in cm
  4. Petal width in cm
- 150 samples



"a genius who almost single-handedly  
created the foundations for modern  
statistical science"

# Data



Ronald Fisher

## Iris dataset

- Introduced by the British statistician and biologist Ronald Fisher in his 1936 paper "The use of multiple measurements in taxonomic problems"
- Collected data to quantify the morphologic variation of Iris flowers of three related species

# Classes



*Iris setosa*



*Iris versicolor*



*Iris virginica*

# Features

## Iris dataset

- Four features were measured from each sample: the length and the width of the sepals and petals, in centimeters
- Based on the combination of these four features, Fisher developed a model to distinguish the species from each other.

# The iris dataset

Datafile Name: Fisher's Iris

Datafile Subjects: Agriculture , Famous datasets

Story Names: Fisher's Irises

Reference: Fisher, R. A. (1936). The Use of Multiple Measurements in Axonomic Problems. *Annals of Eugenics* 7, 179-188.

Authorization: free use

Description: This is a dataset made famous by Fisher, who used it to illustrate principles of discriminant analysis. It contains 6 variables with 150 observations.

Number of cases: 150

Variable Names:

1.Species\_No: Flower species as a code

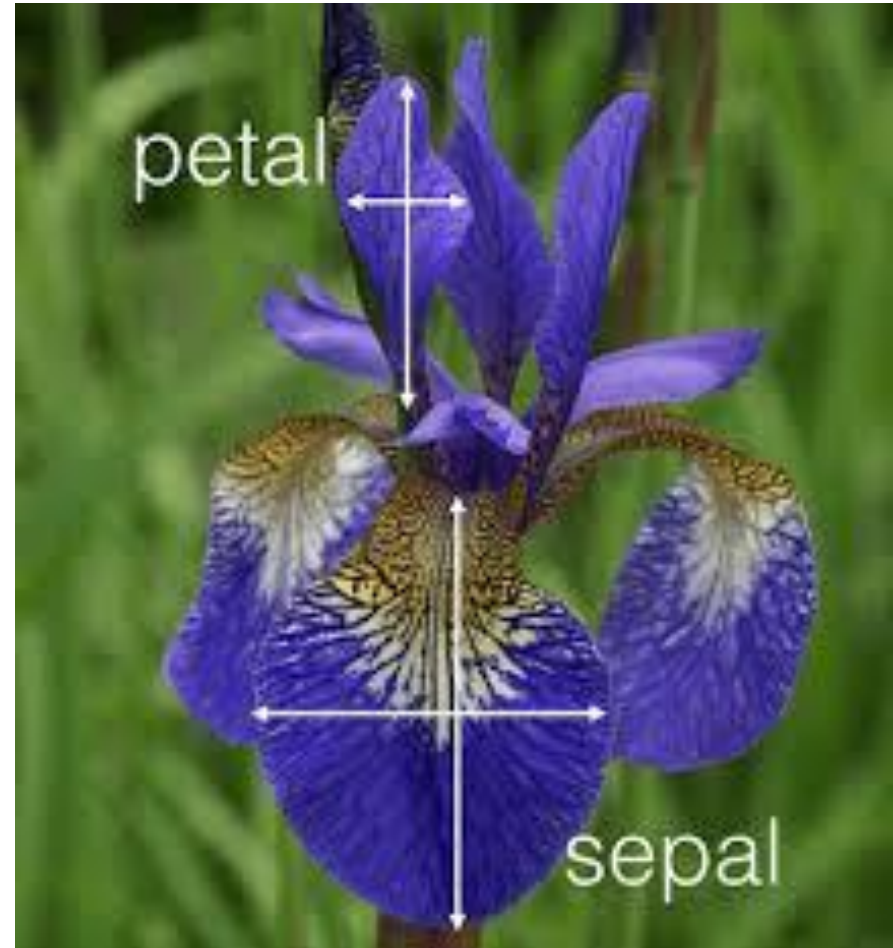
2.Species\_Name: Species name

**3.Petal\_Width: Petal Width**

**4.Petal\_Length: Petal Length**

**5.Sepal\_Width: Sepal Width**

**6.Sepal\_Length: Sepal Length**



# The Iris dataset raw data



Iris Setosa



Iris Virginica



Iris Versicolor

Meaning of the 3 classes

|   | Sepal.Length | Sepal.Width | Petal.Length | Petal.Width | Species |
|---|--------------|-------------|--------------|-------------|---------|
| 1 | 5.1          | 3.5         | 1.4          | 0.2         | setosa  |
| 2 | 4.9          | 3.0         | 1.4          | 0.2         | setosa  |
| 3 | 4.7          | 3.2         | 1.3          | 0.2         | setosa  |
| 4 | 4.6          | 3.1         | 1.5          | 0.2         | setosa  |
| 5 | 5.0          | 3.6         | 1.4          | 0.2         | setosa  |
| 6 | 5.4          | 3.9         | 1.7          | 0.4         | setosa  |
| 7 | 4.6          | 3.4         | 1.4          | 0.3         | setosa  |
| 8 | 5.0          | 3.4         | 1.5          | 0.2         | setosa  |

⋮

# Y = FUNC( X )

## that's it!!!

|   | Sepal.Length | Sepal.Width | Petal.Length | Petal.Width | Species |
|---|--------------|-------------|--------------|-------------|---------|
| 1 | 5.1          | 3.5         | 1.4          | 0.2         | setosa  |
| 2 | 4.9          | 3.0         | 1.4          | 0.2         | setosa  |
| 3 | 4.7          | 3.2         | 1.3          | 0.2         | setosa  |
| 4 | 4.6          | 3.1         | 1.5          | 0.2         | setosa  |
| 5 | 5.0          | 3.6         | 1.4          | 0.2         | setosa  |
| 6 | 5.4          | 3.9         | 1.7          | 0.4         | setosa  |
| 7 | 4.6          | 3.4         | 1.4          | 0.3         | setosa  |
| 8 | 5.0          | 3.4         | 1.5          | 0.2         | setosa  |

- For every dataset in machine learning or toolbox, is all about to create X and Y

• X

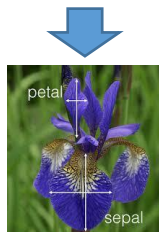
Sample, sample, .....

|          |     |  |  |  |  |  |  |  |
|----------|-----|--|--|--|--|--|--|--|
| Features | 5.1 |  |  |  |  |  |  |  |
|          | 3.5 |  |  |  |  |  |  |  |
|          | 1.4 |  |  |  |  |  |  |  |
|          | 0.2 |  |  |  |  |  |  |  |

• Y

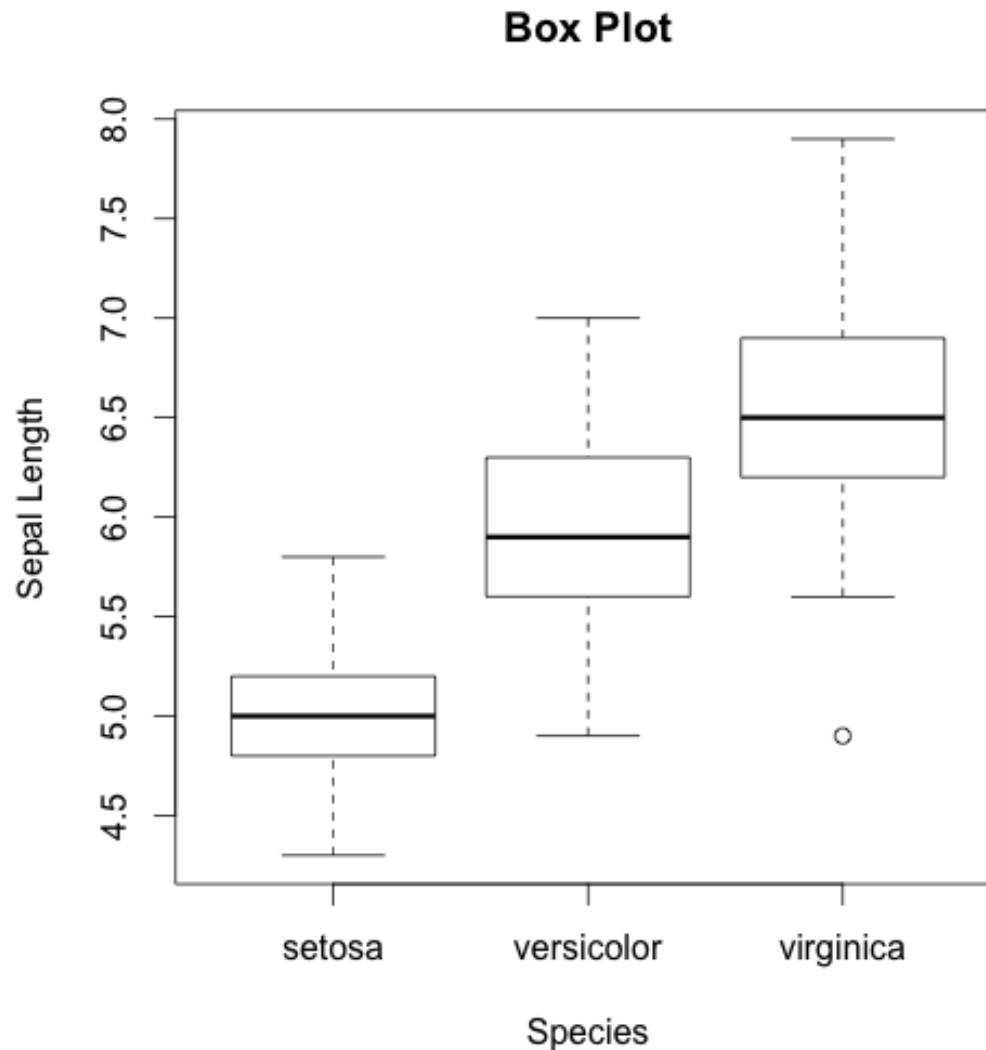
Class ID, or value to be learnt

|   |   |   |   |  |  |  |  |  |
|---|---|---|---|--|--|--|--|--|
| 1 | 1 | 3 | 2 |  |  |  |  |  |
|---|---|---|---|--|--|--|--|--|



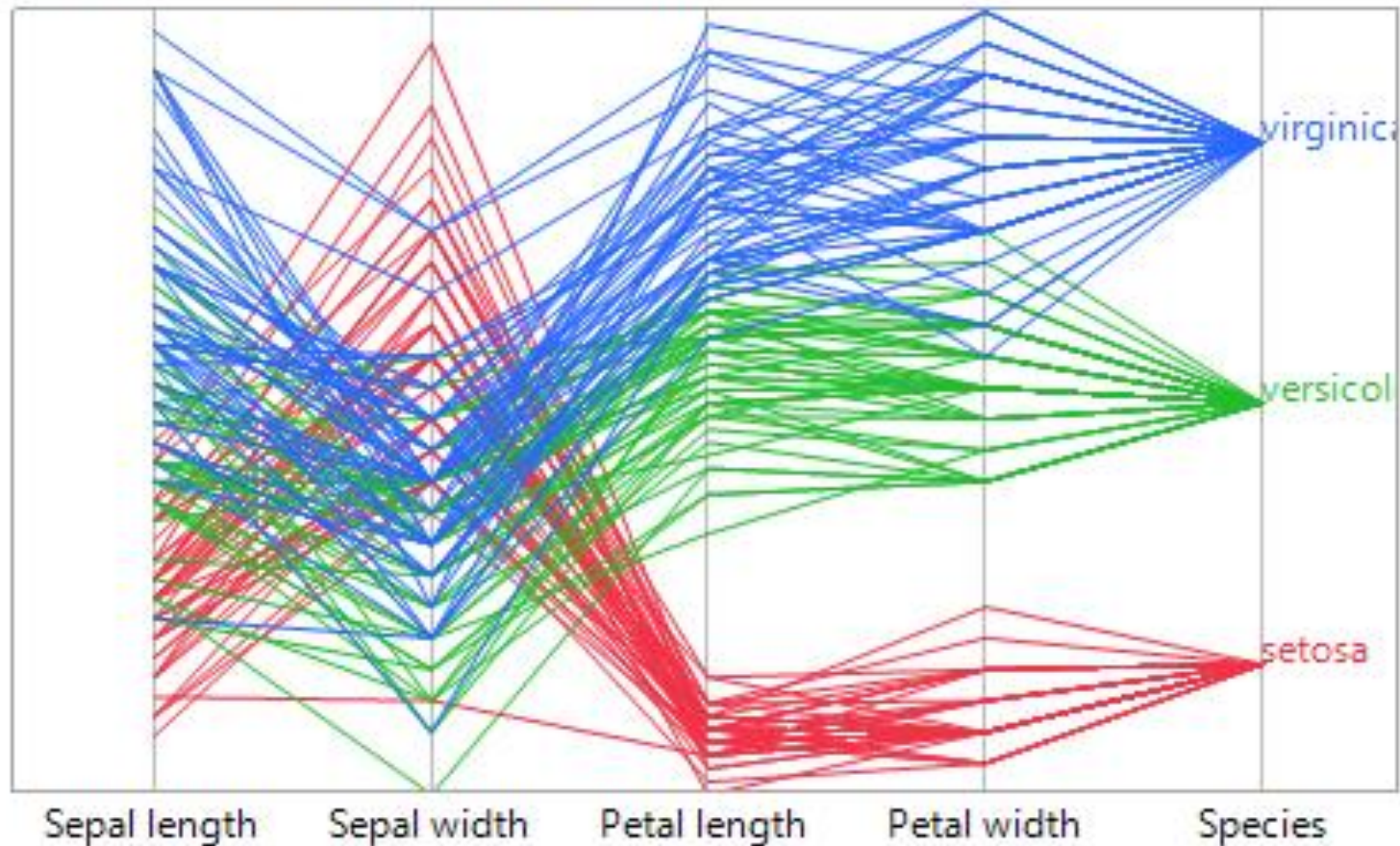
1 = «setosa»

# The Fischer Iris boxplots



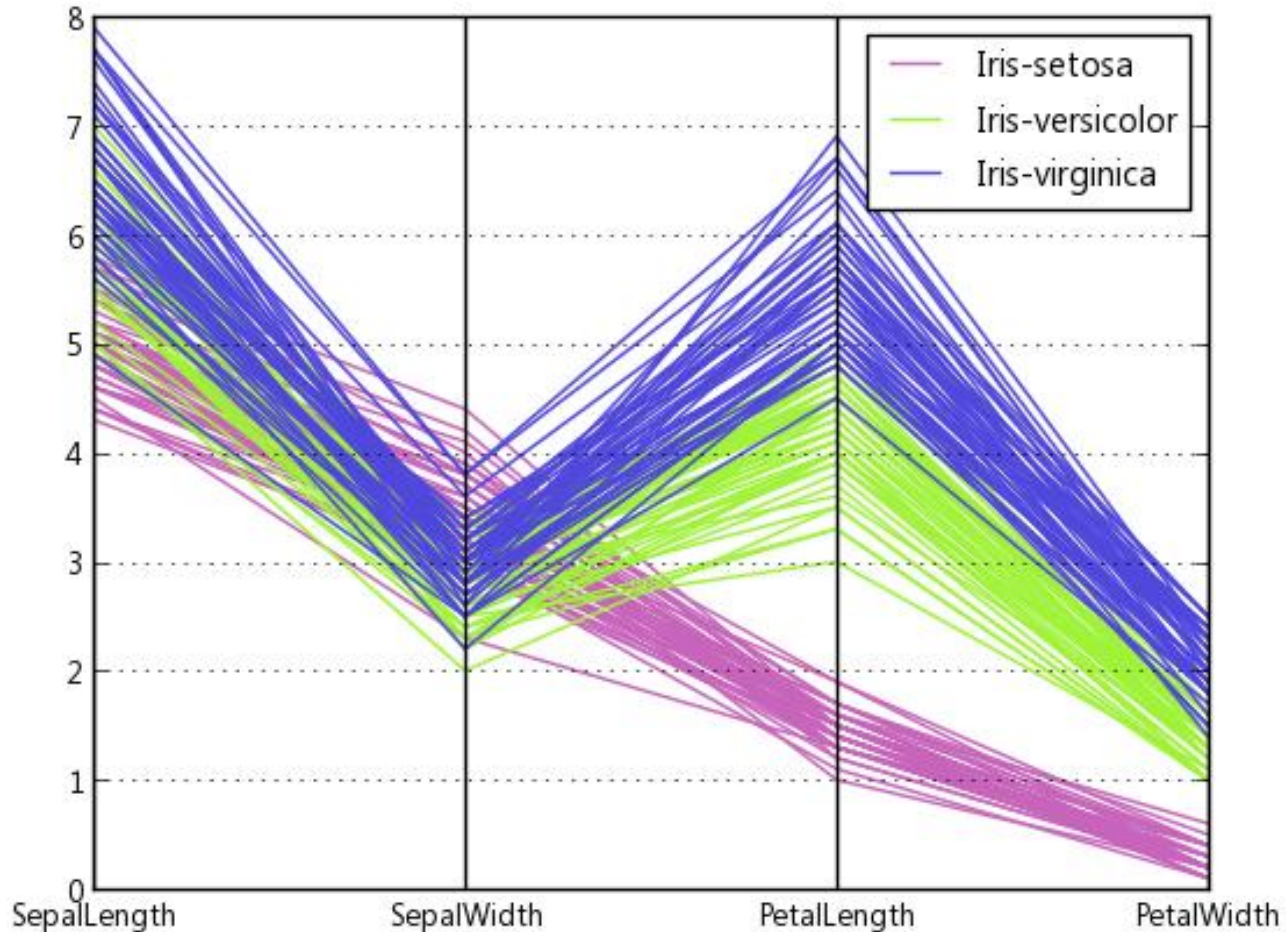
# Iris Data – Parallel Plots

**Parallel Plot - Look for Patterns**



# Using a different toolbox

Note:  
you need  
same units  
or a similar  
scale here!!

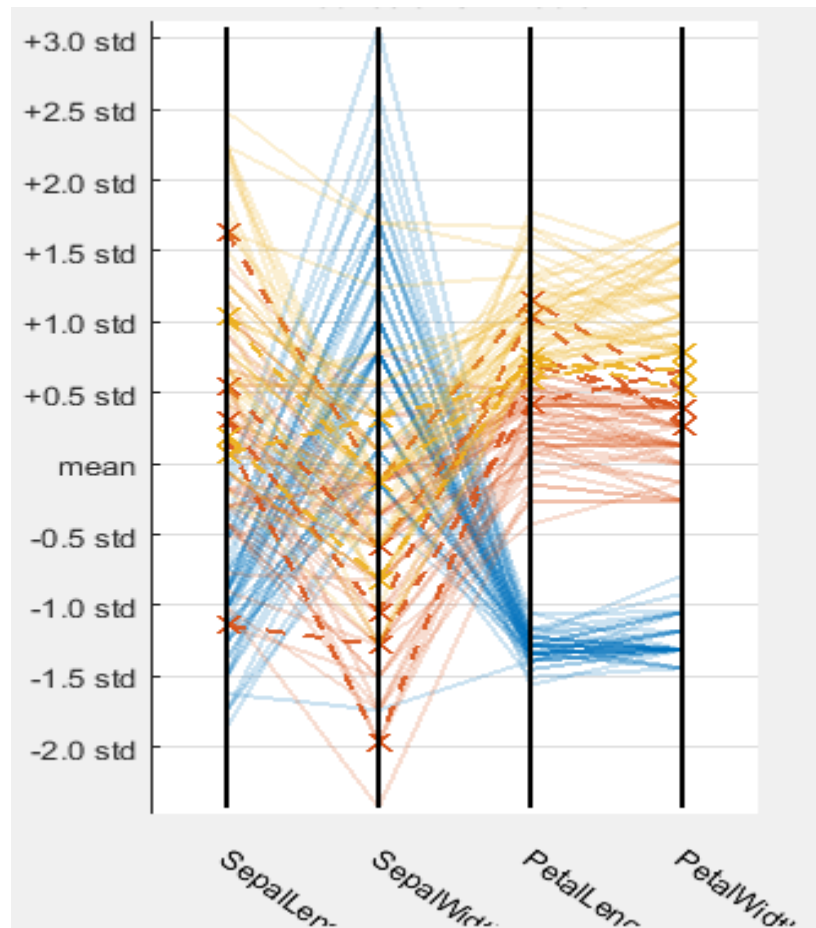


# Harmonization

Using the  
**mean** and the  
**std** fractions  
you can  
harmonize the  
values

(z score)

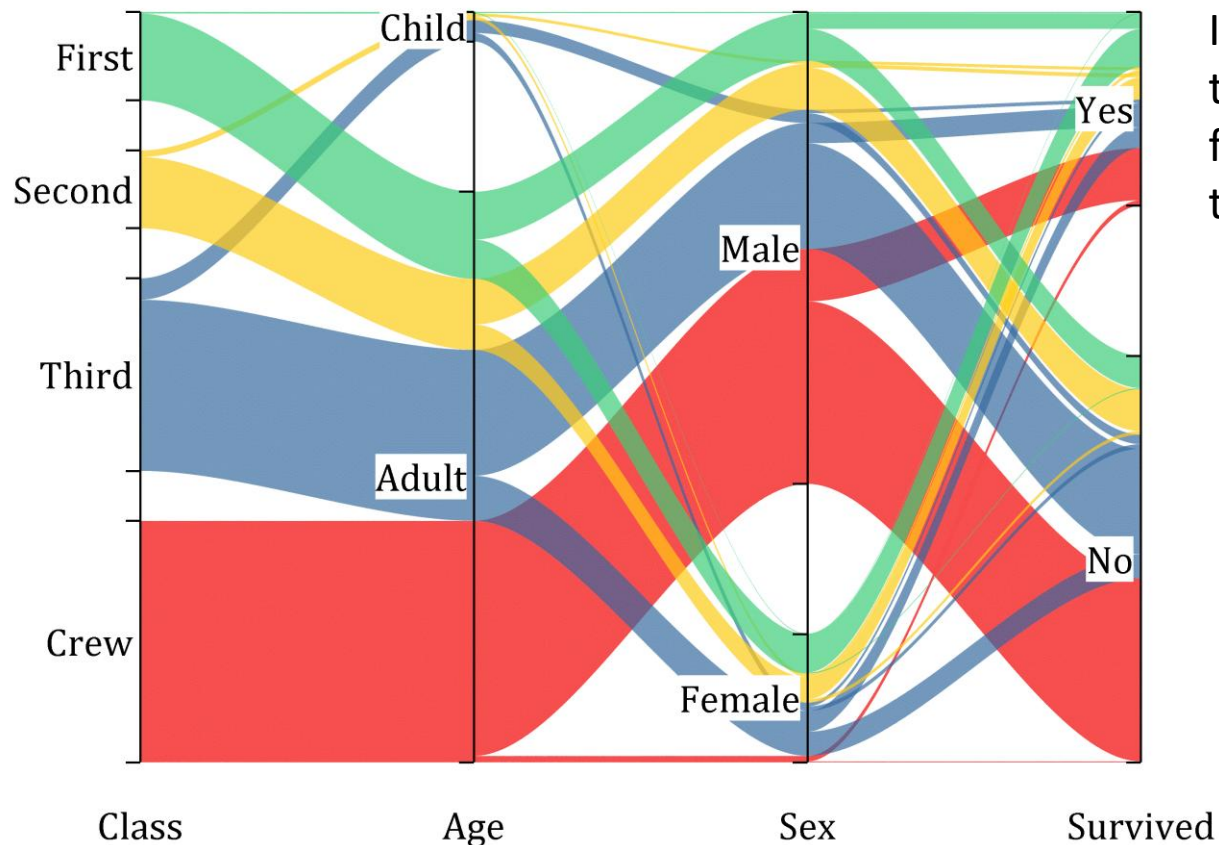
mean of all of the  
values is 0 and the  
standard deviation is 1



## Classes

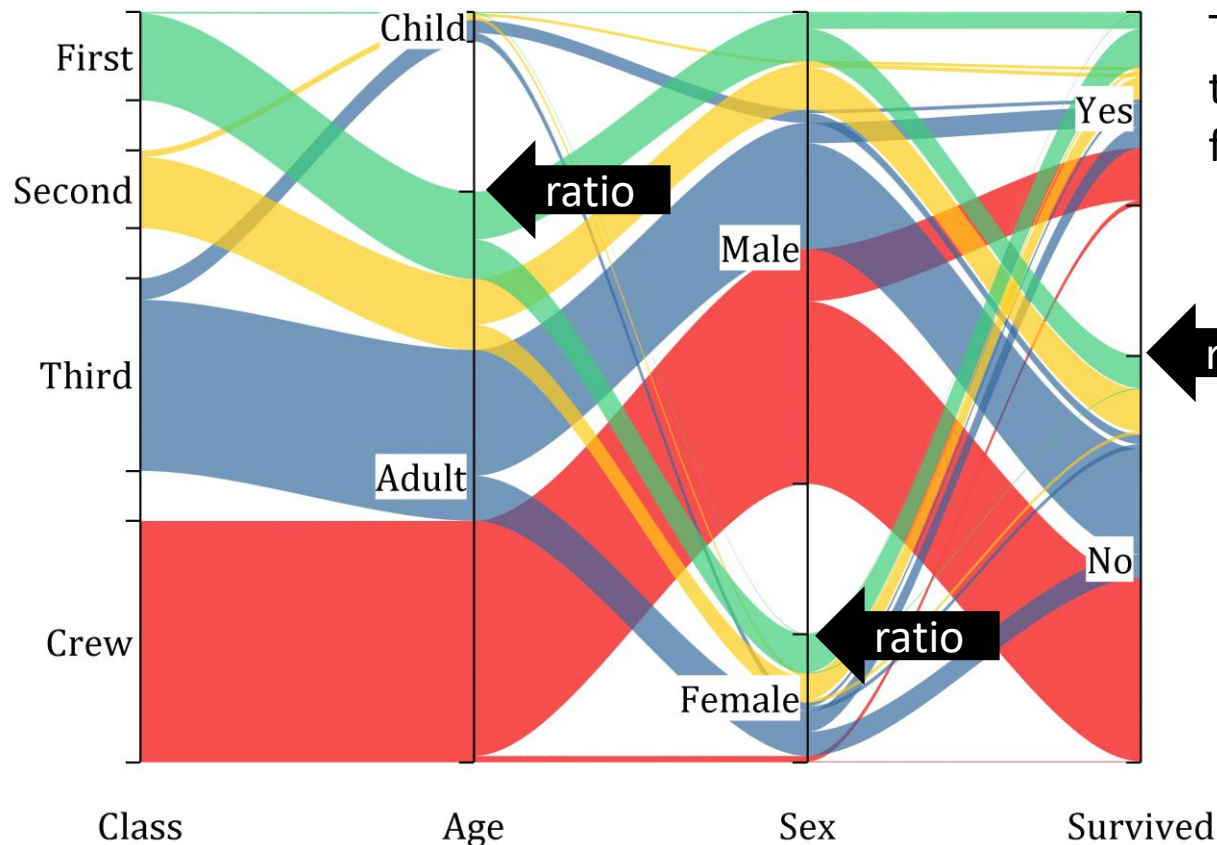
|            |
|------------|
| setosa     |
| versicolor |
| virginica  |

# EDA, Parallel Plot: a more complex case



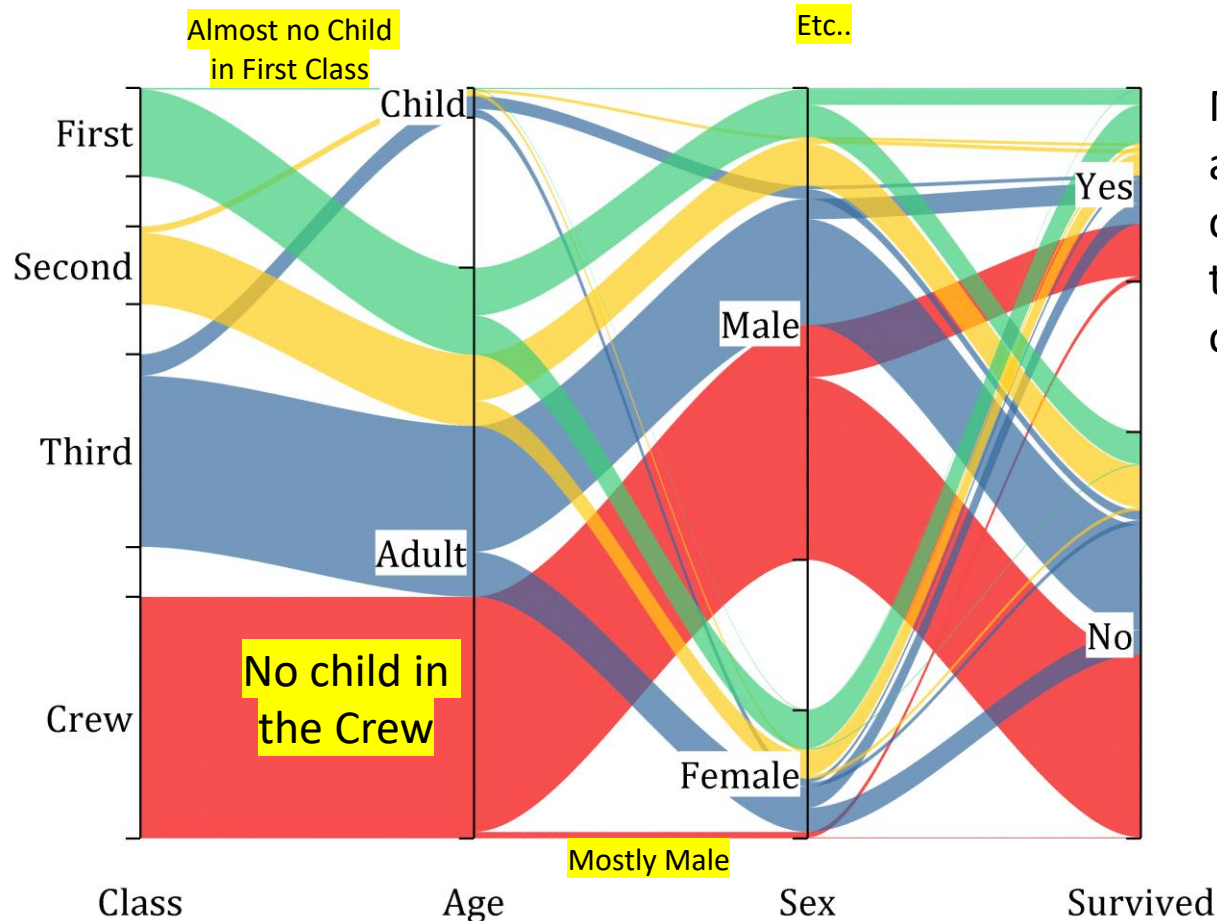
In this case it is harder to understand which feature will better help the classification

# EDA, Parallel Plot: a more complex case



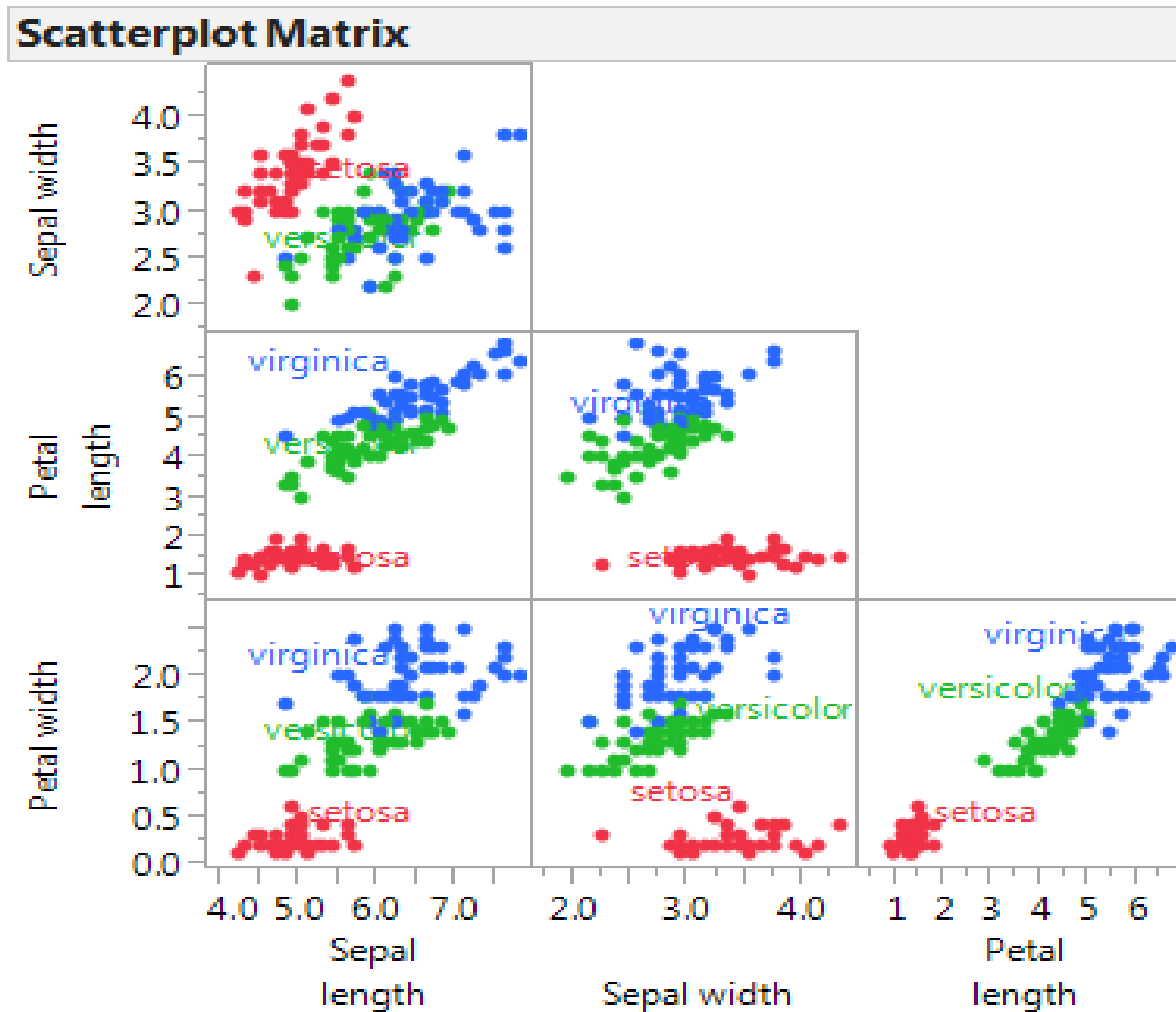
The plot is showing also the ratio between the feature (crisp) values

# EDA, Parallel Plot: a more complex case



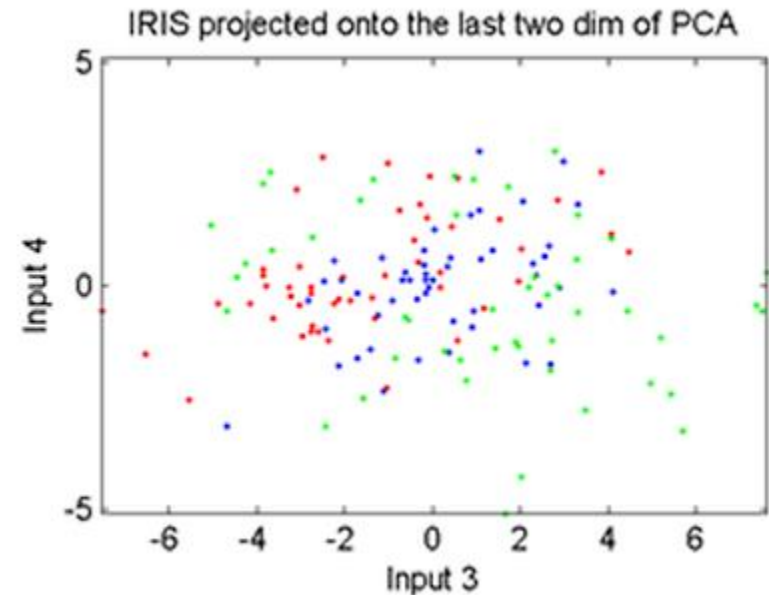
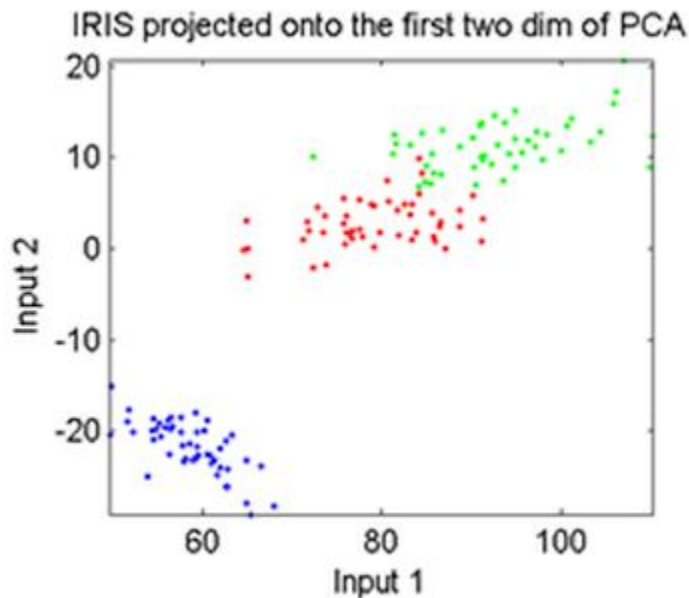
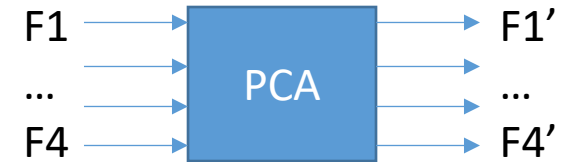
Meaningful information and valuable insights can be extracted from this plot just by observation

# Iris Data - Correlation Plots



# Principal Component Analysis on the Iris Dataset

The PCA is an effective method for **reducing** a dataset's **dimensionality** while keeping spatial characteristics as much as possible (no need for labeled data)



# Data harmonization

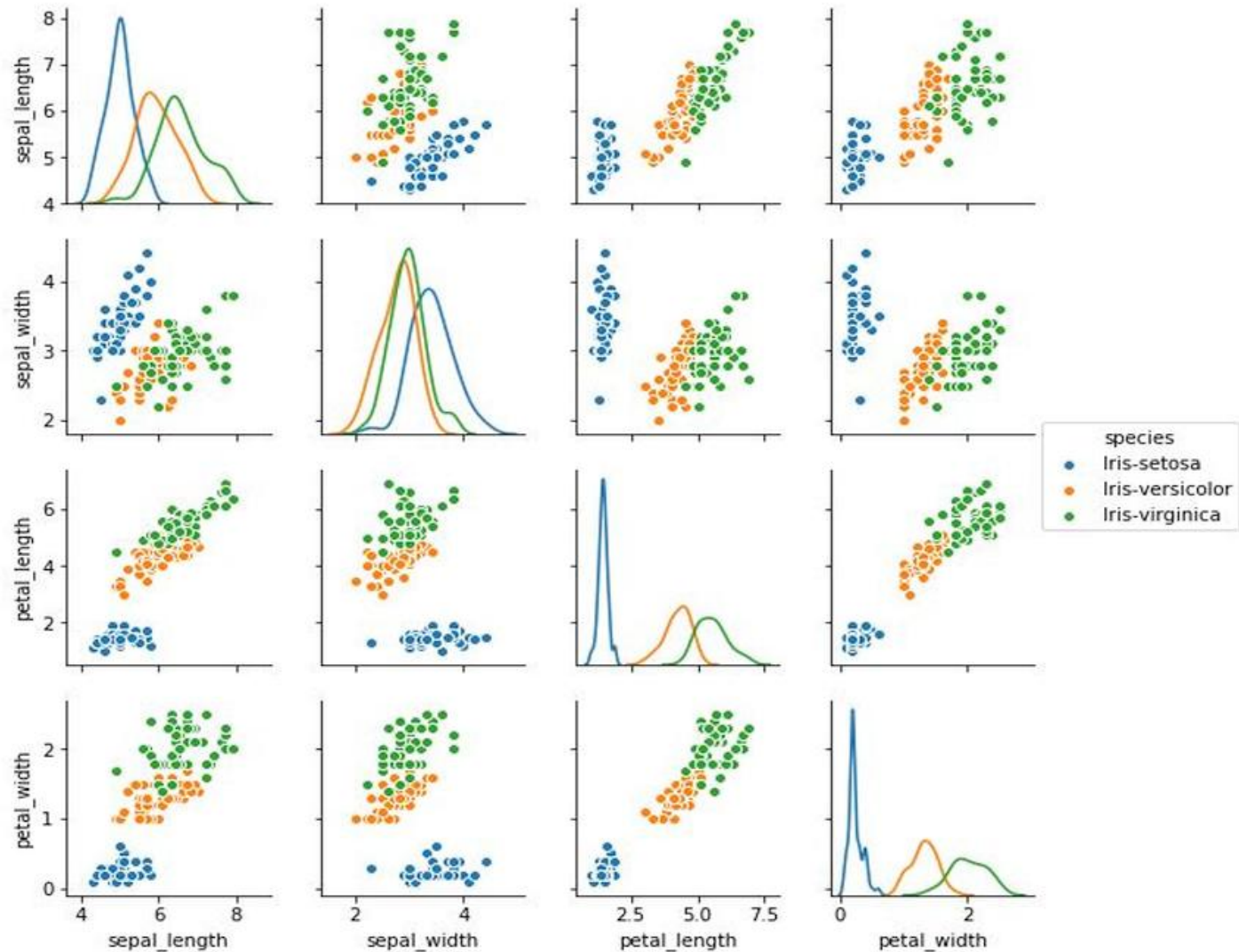
- No missing data
- No labelling errors
- No scaling needed
- WE WILL USE THE FISCHER IRIS DATASET AS A **DEBUG TOOLS!!**
- Let's use the Google **TensorFlow** notation in the next examples
- **TensorFlow** is a free and open-source software library for dataflow and differentiable programming across a range of tasks from Google.



```
#Plot the dataset seaborn.pairplot(dataset, hue="species", size=2, diag_kind="kde") plt.show()
```

Fabio Scotti - Università degli Studi di Milano

# The (pair)plot



# Getting ready to start learning

1. Encoding
  1. Integer encoding
  2. One-Hot encoding
2. Prepare the Input Features
3. Split Train Test
4. **TensorFlow** Neural Network



# Clustering and Classification with Categorical data

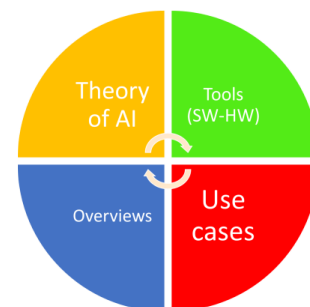
How to apply the supervised and unsupervised techniques presented in the course when the input datum in the input vectors is not a numerical value



# THEORY

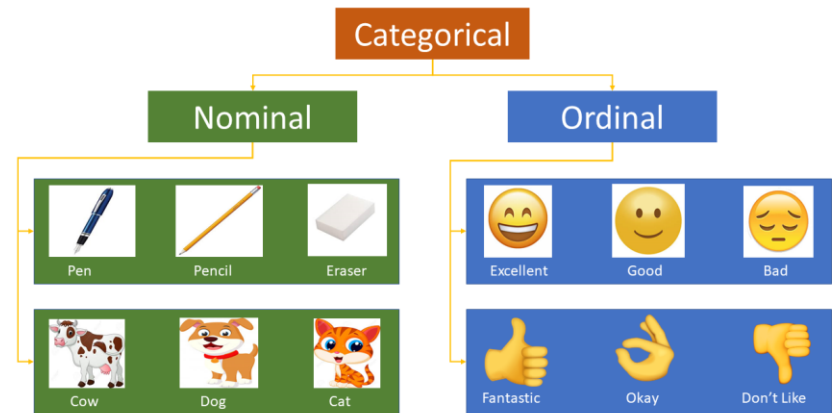
## Integer encoding one-hot encoding

Learning can be affected by the way you list the classes



# Categorical Features in IS4

- Industrial dataset rarely contain only numerical values. Logs and events often include categorical attributes that must be handled carefully.



## Examples in industrial and maintenance applications

| Feature            | Possible categories                                            |
|--------------------|----------------------------------------------------------------|
| Machine type       | pump, motor, conveyor, compressor, robot arm                   |
| Failure type       | bearing wear, overheating, leakage, misalignment, sensor fault |
| Alarm code         | A17, T02, VIB_HIGH, TEMP_WARN, PRESS_LOW                       |
| Maintenance action | inspection, lubrication, replacement, recalibration, cleaning  |
| Operator shift     | morning, afternoon, night                                      |
| Production line    | line 1, line 2, line 3                                         |
| Component status   | normal, warning, critical, offline                             |
| Supplier           | supplier A, supplier B, supplier C                             |
| Material type      | steel, aluminum, plastic, composite                            |
| Product family     | standard, customized, prototype, spare part                    |

# Some feature (ordinal) can be mapped in integers

## Categorical → Numerical via **INTEGER ENCODING**

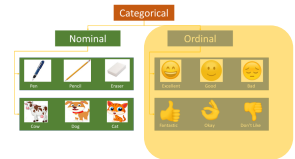
Not all categorical features are equal. Some categories represent an **ordered scale**, where mapping values to integers is meaningful.

### Ordinal features

- Categories have a **natural ordering**
- Differences reflect increasing or decreasing intensity
- Integer values preserve semantic relationships

### Examples

- **Priority level:**
  - low → 1
  - medium → 2
  - high → 3
- **Severity or criticality:**
  - informational → 1
  - warning → 2
  - critical → 3
- **Trust or risk level:**
  - trusted < unknown < suspicious



# Categorical → Numerical via INTEGER ENCODING

## Why integer encoding is valid here

- Order is semantically meaningful
- Higher values represent stronger effect or importance
- Distance between values has an interpretable meaning

### Integer Encoding

#### *Definition*

A feature transformation technique that maps categorical values to integer numbers, preserving an explicit ordering when the categories represent an ordinal scale

# Categorical → Numerical via LABEL ENCODING

Label encoding assigns a unique integer to each distinct categorical value.

This example shows how destination IP addresses can be encoded as labels.

## What label encoding does

- Assigns a unique integer identifier to each IP address
- Preserves identity, **not semantics**
- Introduces an **artificial numerical ordering** (can be bad in some training method!)

From IOT mesh network

**dst\_ip**

-----

**192.168.1.10**

**10.0.0.5**

**172.16.0.3**

**192.168.1.10**

192.168.1.10 → 0

10.0.0.5 → 1

172.16.0.3 → 2

# When label Encoding can be used

## Acceptable scenarios

- Tree-based models (Decision Trees, Random Forests)
- Hashing or indexing purposes
- As an intermediate step before embeddings

## Not recommended for

- K-Means
- DBSCAN
- Any distance-based or linear model

### "Lazy Encoding"

Label encoding is arbitrary....

#### EXAMPLE: ENCODING IPs

It masks the hierarchical nature of IP addresses.

"The fundamental challenge is that IPs act as **categorical identifiers** but follow a **hierarchical logic** (network/subnet/host) that standard encoding often fails to capture."

The model must work harder (require more splits) to "rediscover" that 192.168.1.1 and 192.168.1.2 are related.  
**YOU HAVE TO ENCODE WISELY!!!**

# One Hot Encoding

- **Why Encoding is Required**
- Categorical values have **no natural ordering**
- Direct numeric mapping (e.g. tcp=1, udp=2) introduces **artificial distances**
- Distance computations would become **meaningless**

protocol = {tcp, udp, icmp}  
tcp=[1,0,0],udp=[0,1,0],icmp=[0,0,1]

| Label Encoding |               |          | One Hot Encoding |         |          |          |
|----------------|---------------|----------|------------------|---------|----------|----------|
| Food Name      | Categorical # | Calories | →                |         |          |          |
| Apple          | 1             | 95       | Apple            | Chicken | Broccoli | Calories |
| Chicken        | 2             | 231      | 1                | 0       | 0        | 95       |
| Broccoli       | 3             | 50       | 0                | 1       | 0        | 231      |
|                |               |          | 0                | 0       | 1        | 50       |
| 1 Feature      |               |          | 3 Features       |         |          |          |

## One-Hot Encoding

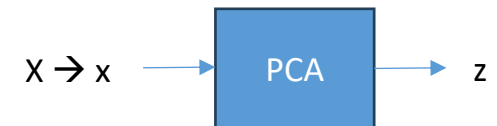
- Each categorical feature with  $C$  possible values is mapped to  $C$  binary dimensions
- Exactly one dimension is active (value = 1)

# After One-Hot Encoding

- One-Hot Encoding enables:
  - valid distance computation
  - meaningful covariance estimation
  - Dimensionality increase is **expected and intentional**
- PCA can be applied **after encoding** to:
  - reduce dimensionality
  - remove redundancy
  - enable visualization

| Label Encoding |               |          | One Hot Encoding |         |          |          |
|----------------|---------------|----------|------------------|---------|----------|----------|
| Food Name      | Categorical # | Calories | Apple            | Chicken | Broccoli | Calories |
| Apple          | 1             | 95       | 1                | 0       | 0        | 95       |
| Chicken        | 2             | 231      | 0                | 1       | 0        | 231      |
| Broccoli       | 3             | 50       | 0                | 0       | 1        | 50       |

1 Feature → 3 Features



# Dummy Encoding

**Dummy encoding** is a technique used to convert a categorical variable with  $C$  possible values into **binary numerical variables**.

- Each category is represented by a binary indicator (0 or 1), where:
- **1** indicates the presence of the category
- **0** indicates its absence
- For a categorical feature with  $C$  categories, dummy encoding produces  $C$  (or  $C - 1$ ) binary features.

The diagram illustrates the transformation of a categorical variable  $X$  into binary features. It starts with a table of  $id$  and  $X$  values. Two arrows branch from this table: one labeled 'One-Hot Encoding' pointing to a table with three columns ( $X=a$ ,  $X=b$ ,  $X=c$ ), and another labeled 'Dummy Encoding' pointing to a table with two columns ( $X=a$ ,  $X=b$ ). The 'One-Hot Encoding' table has 8 rows, each with a 1 in exactly one of the three columns. The 'Dummy Encoding' table also has 8 rows, but the last row (id=8) has a 1 in the  $X=b$  column and 0s elsewhere, which is inconsistent with the text '...It means c' next to it. The text '...It means c' appears three times next to the last three rows of the Dummy Encoding table.

| id | X |
|----|---|
| 1  | a |
| 2  | c |
| 3  | a |
| 4  | b |
| 5  | a |
| 6  | c |
| 7  | c |
| 8  | b |

| id | X = a | X = b | X = c |
|----|-------|-------|-------|
| 1  | 1     | 0     | 0     |
| 2  | 0     | 0     | 1     |
| 3  | 1     | 0     | 0     |
| 4  | 0     | 1     | 0     |
| 5  | 1     | 0     | 0     |
| 6  | 0     | 0     | 1     |
| 7  | 0     | 0     | 1     |
| 8  | 0     | 1     | 0     |

| id | X = a | X = b |
|----|-------|-------|
| 1  | 1     | 0     |
| 2  | 0     | 0     |
| 3  | 1     | 0     |
| 4  | 0     | 1     |
| 5  | 1     | 0     |
| 6  | 0     | 0     |
| 7  | 0     | 0     |
| 8  | 0     | 1     |

...It means c

...It means c

...It means c

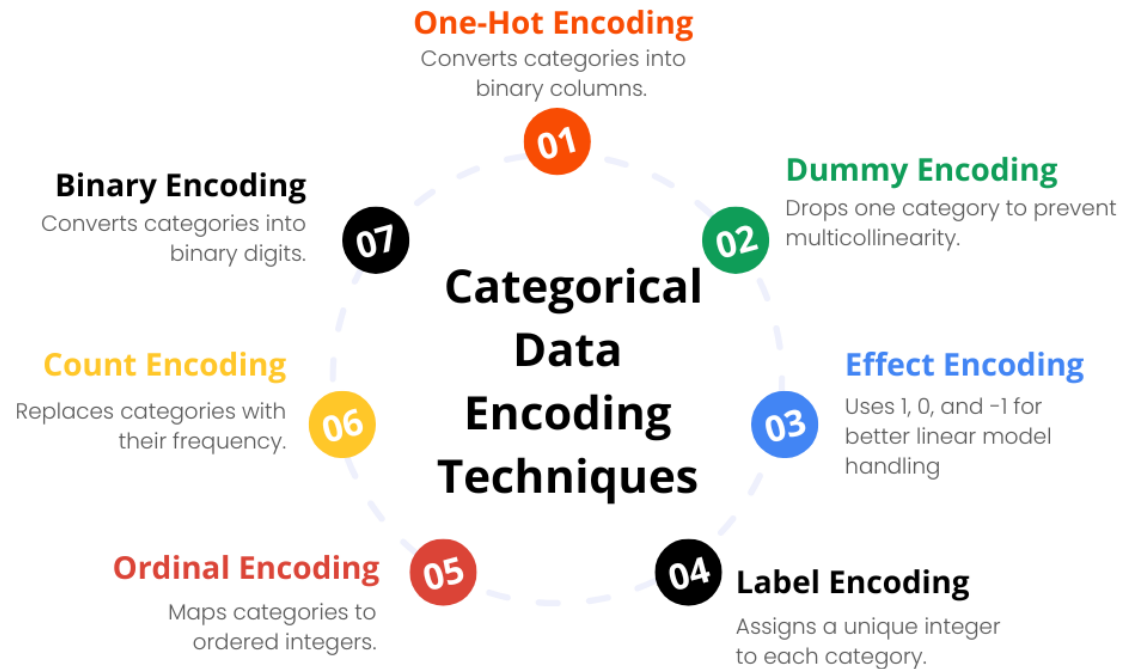
# Integer encoding

- Each unique category value is assigned an integer value. For example,
  - “*red*” is 1,
  - “*green*” is 2,
  - and “*blue*” is 3.
- This is called a label encoding or **integer encoding** and is easily reversible.
- For some variables, this may be enough.
  - The integer values have a natural ordered relationship between each other, and machine learning algorithms may be able to understand and harness this relationship.



# Other encoding method for Categorical data

- There are many more techniques to encode the categorical data

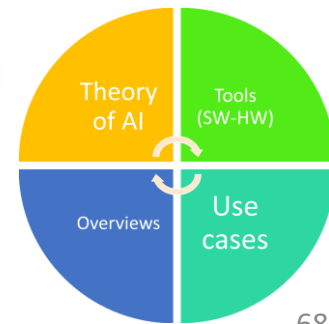




# Toolboxes

## Coding Normalization and Train/Test partitions

The basic tools to start



# One Hot Encoding in Matlab

```
labels = ["red"; "blue"; "red"; "green"; "yellow"; "blue"]; % 6 elements
labels = categorical(labels); % 4 only different labels
categories(labels)
ans = 4x1 cell
    {'blue' }
    {'green' }
    {'red'   }
    {'yellow'}
labels = onehotencode(labels,2)
labels = 6x4
```

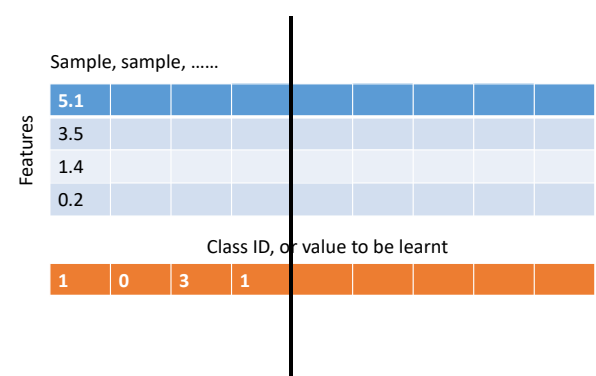
|   |   |   |   |
|---|---|---|---|
| 0 | 0 | 1 | 0 |
| 1 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 0 |
| 0 | 0 | 0 | 1 |
| 1 | 0 | 0 | 0 |

# Prepare the Input Features

- To improve gradient descent, we will normalize the values utilizing the `normalize()`
- `X_data` variable will contain our normalized features we will use to train our neural network.

```
from sklearn.preprocessing import normalize
FEATURES = dataset.columns[0:4]
X_data = dataset[FEATURES].as_matrix() #for compatib.
X_data = normalize(X_data)
```

# Split Train Test



The diagram illustrates a data matrix split into two parts by a vertical line. The top part, labeled 'Features', has 4 rows with values 5.1, 3.5, 1.4, and 0.2. The bottom part, labeled 'Class ID, or value to be learnt', has 4 rows with values 1, 0, 3, and 1. The columns are labeled 'Sample, sample, .....'. The first four columns are blue, and the last four columns are orange.

| Sample, sample, ..... | Sample, sample, ..... | Sample, sample, ..... | Sample, sample, ..... | Sample, sample, ..... | Sample, sample, ..... | Sample, sample, ..... | Sample, sample, ..... |
|-----------------------|-----------------------|-----------------------|-----------------------|-----------------------|-----------------------|-----------------------|-----------------------|
| 5.1                   |                       |                       |                       |                       |                       |                       |                       |
| 3.5                   |                       |                       |                       |                       |                       |                       |                       |
| 1.4                   |                       |                       |                       |                       |                       |                       |                       |
| 0.2                   |                       |                       |                       |                       |                       |                       |                       |
| 1                     | 0                     | 3                     | 1                     |                       |                       |                       |                       |

- The 2 vectors, X\_data and y, contains the data needed to train a neural network with the engine (like Tensorflow).
- Now split this data into a training and a test set in order to prevent overfitting and be able to obtain a better benchmark of our network's performance.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X_data, Y, test_size=0.3,
random_state=1)
```

Use this parameter wisely in case of time-series

# Split Train Test

| Sample, sample, .....           |     |   |   |  |  |  |  |  |  |
|---------------------------------|-----|---|---|--|--|--|--|--|--|
| Features                        | 5.1 |   |   |  |  |  |  |  |  |
|                                 | 3.5 |   |   |  |  |  |  |  |  |
|                                 | 1.4 |   |   |  |  |  |  |  |  |
|                                 | 0.2 |   |   |  |  |  |  |  |  |
| Class ID, or value to be learnt |     |   |   |  |  |  |  |  |  |
| 1                               | 0   | 3 | 1 |  |  |  |  |  |  |

```
import numpy as np
from sklearn.model_selection import train_test_split
```

```
[53] X1 = np.arange(10)
      X = np.reshape(X1, (5, 2))
      X
```

```
array([[0, 1],
       [2, 3],
       [4, 5],
       [6, 7],
       [8, 9]])
```

```
[54] y = range(5)
      list(y)
```

```
[0, 1, 2, 3, 4]
```

```
[55] X_train, X_test, y_train, y_test = train_test_split(
      ...     X, y, test_size=0.33, random_state=1)
```

```
[56] X_train
```

```
array([[8, 9],
       [0, 1],
       [6, 7]])
```

```
[57] y_train
```

```
[4, 0, 3]
```

```
array([[0, 1],
       [2, 3],
       [4, 5],
       [6, 7],
       [8, 9]])
```

```
X_test
```

```
array([[4, 5],
       [2, 3]])
```

# Next step is learning

- As you can see in the exercitation in Maltlab and Colab we are now ready to create the first classifiers!



# Main points



- The FISHER IRIS dataset as “debug tool”
- One-hot encoding
- Creating the matrices for learning
- Statistical analysis of the data
- Example relevant of plots and charts for ML
- Statistical analysis is needed before to start processing
- Data Visualization is quite useful....

# LESSON 15

## Part B

Basic datasets management,  
Coding in Matlab and Colab,  
One-hot encoding



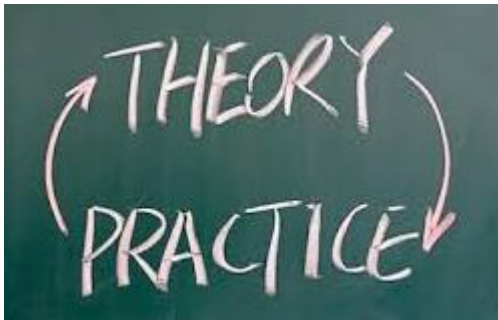
# Laboratory outline

- **LABORATORY (in a separate file):**  
Loading and basic dataset management
  - Colab example
  - Matalab example
- The laboratory is based on the IRIS dataset



# What in the exam?

- In the exam there will be **no (exact) coding**
- Nevertheless, is it important to study the application of the concept of the theory in practical cases and examples in order to better understand the notions



Knowledge

- You will be asked **to comment some example** of code

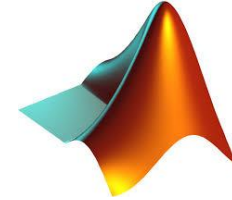
# LAB CLASS

## COLAB



- Environment preparation
- Data loading

## MATLAB



- Load the dataset
- Create the data structures (X and Y)

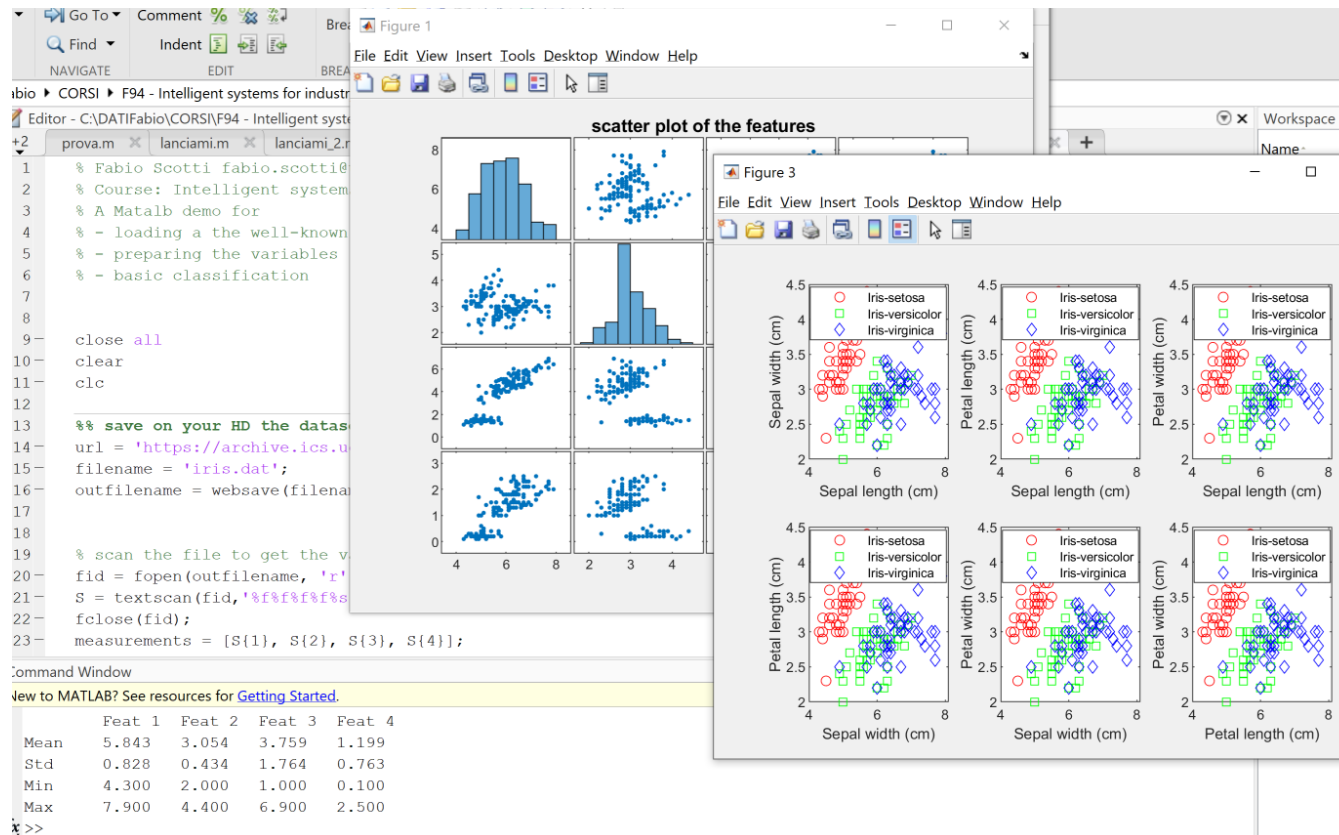
- Preliminary data analysis
- Plot the data
- Normalization
- Classification example

# Best thing to do... (Matlab)

- Download the example and let them run!



laboratory\_matlab\_IRIS\_Dataset.m



The example is commented... explore the code and try to modify it and/or add new plots

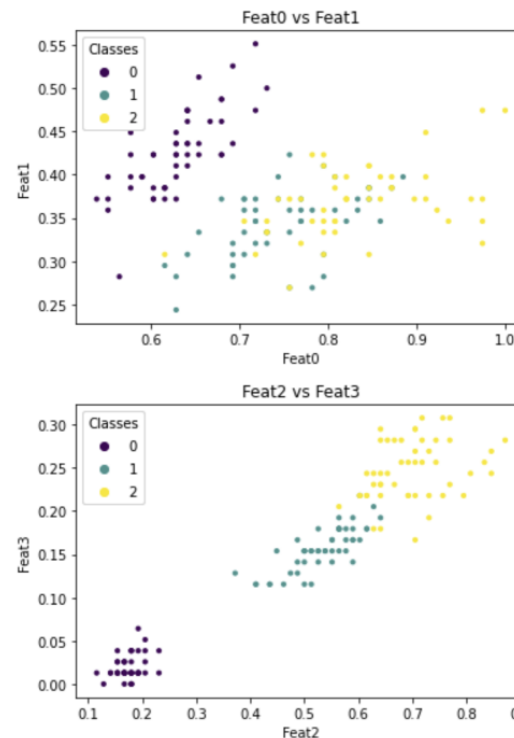
# Best thing to do... (Colab)

- Download the code  `laboratory_COLAB_IRIS_Dataset.ipynb`  
 → upload it to your Gdrive <https://colab.research.google.com/>  
 → open it with Colab and let it run  
 (use Google Chrome – with login)

Plot features and classes

```
[41] import matplotlib.pyplot as plt
# select 2 features to plot
feat0 = xNumpy_Norm[:,0]
feat1 = xNumpy_Norm[:,1]
# params
area = np.pi*3
# Plot
fig1, ax1 = plt.subplots()
scatter1 = ax1.scatter(feat0, feat1, s=area, c=yNumpy)
plt.title('Feat0 vs Feat1')
plt.xlabel('Feat0')
plt.ylabel('Feat1')
legend1 = ax1.legend(*scatter1.legend_elements(),
                    loc="upper left", title="Classes")
ax1.add_artist(legend1)
plt.show()

# select 2 features to plot
feat2 = xNumpy_Norm[:,2]
feat3 = xNumpy_Norm[:,3]
# params
area = np.pi*3
# Plot
fig2, ax2 = plt.subplots()
scatter2 = ax2.scatter(feat2, feat3, s=area, c=yNumpy)
```

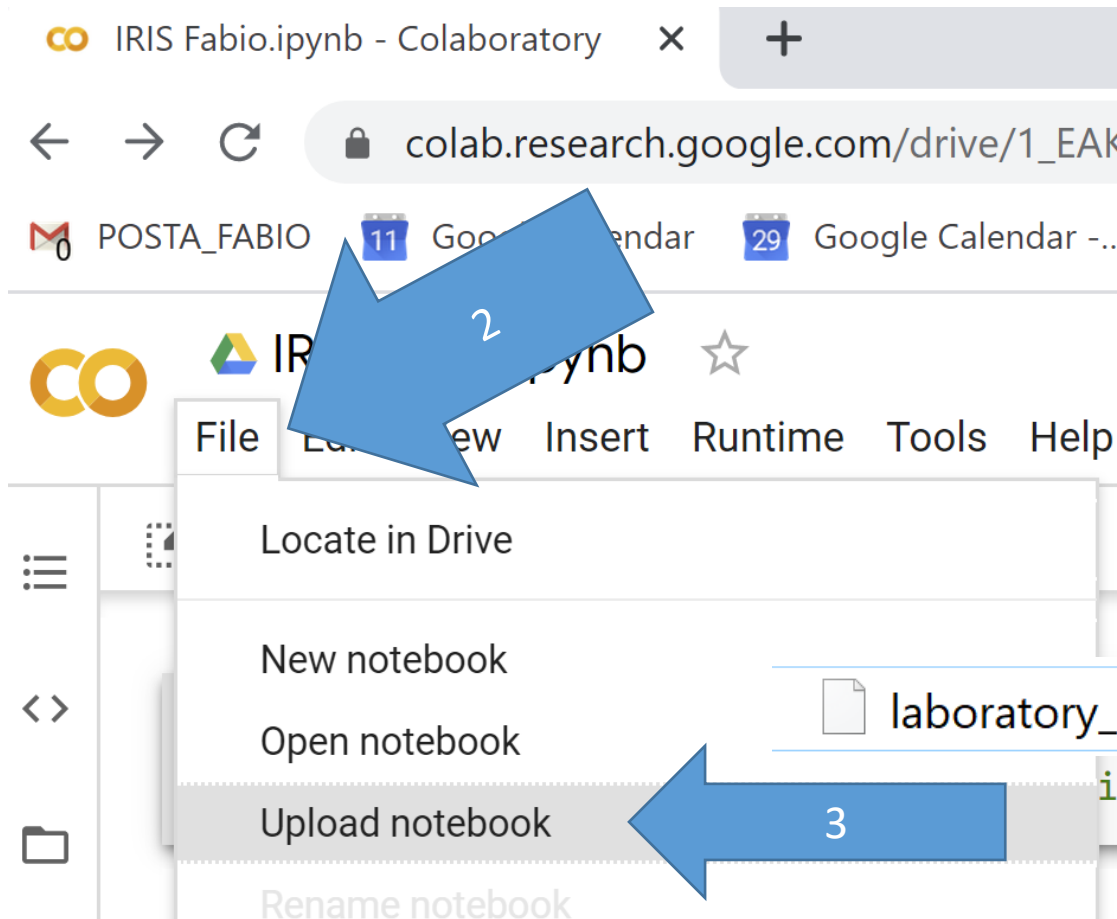
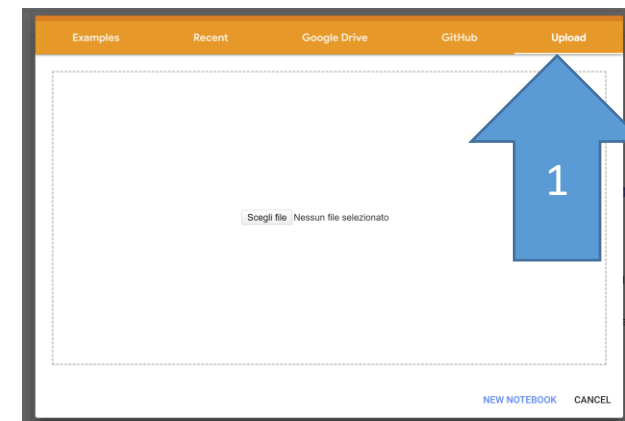


The examples  
are commented

Explore them  
and try to modify  
them  
or to add new  
plots

# Upload your \*.ipynb

<https://colab.research.google.com/>



An **IPYNB** file is a notebook document used by Jupyter Notebook, an interactive computational environment designed to help designers to work with the Python language and their data.

# Let's it run

laboratory\_COLAB\_IRIS\_Dataset.ipynb ☆

File Edit View Insert Runtime Tools Help Last saved at 10:39 PM

+ Code + Text

IRIS data loading, preparation, classification

Fabio Scotti [fabio.scotti@unimi.it](mailto:fabio.scotti@unimi.it)

Course: Intelligent systems for industry, supply chain and environment

A Matlab demo for

- loading a the IRIS dataset
- preparing the variables

[ ]

Import libraries

```
[ ] import pandas as pd #Python Data Analysis Library
import numpy as np #Python Scientific Library
from matplotlib import pyplot as plt
```

Load data

```
[ ] url = "https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data"
new_names = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width', 'iris_class']
dataset = pd.read_csv(url, names=new_names, skiprows=0, delimiter=',')
dataset.info()
```

1) Read the comments

1) Read the comments

## Import libraries

```
[ ] import pandas
import numpy as np
from matplotlib import pyplot as plt
```

2



Import libraries

```
[ ] import pandas as pd #Python Data Analysis Library
import numpy as np #Python Scientific Library
from matplotlib import pyplot as plt
```

**Klik on the [ ]  
to run the cell**

COLAB\_IRIS\_Dataset.ipynb ☆

Insert Runtime Tools Help

Run all Ctrl+F9

Run before Ctrl+F8

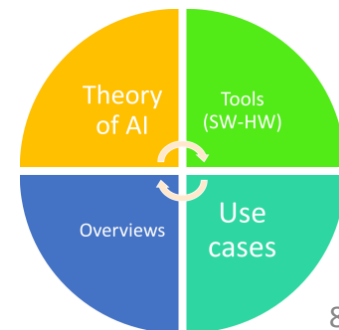
**Or «Run all»**



# Toolboxes

## Loading, EDA/plotting in **Python/Colab**

And let's have a look about **Matplotlib**



# Environment preparation

**Panda:** libraries for dataset preprocessing

**Numpy:** libraries for array manipulation

Import libraries

```
[74] import pandas as pd #Python Data Analysis Library  
import numpy as np #Python Scientific Library
```

NumPy (pronounced /'nʌmpaɪ/ (NUM-py) or sometimes /'nʌmpi/ (NUM-pee)) is a library for the Python programming language, adding support for large, multi-dimensional arrays and matrices, along with a large collection of high-level mathematical functions to operate on these arrays.

# Data loading

## Iris dataset

- Use the panda library to read the CSV

Load data

```
[75] url = "https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data"  
     new_names = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width', 'iris_class']  
     dataset = pd.read_csv(url, names=new_names, skiprows=0, delimiter=',')  
     dataset.info()
```

# Data loading

`dataset.info()` gives us organized information

```
↳ <class 'pandas.core.frame.DataFrame'>  
RangeIndex: 150 entries, 0 to 149  
Data columns (total 5 columns):  
#   Column          Non-Null Count  Dtype  
---  -  
0   sepal_length    150 non-null    float64  
1   sepal_width     150 non-null    float64  
2   petal_length    150 non-null    float64  
3   petal_width     150 non-null    float64  
4   iris_class      150 non-null    object  
dtypes: float64(4), object(1)  
memory usage: 6.0+ KB
```

# Data loading

`dataset.head(n)` to view the first  $n$  samples



|   | sepal_length | sepal_width | petal_length | petal_width | iris_class  |
|---|--------------|-------------|--------------|-------------|-------------|
| 0 | 5.1          | 3.5         | 1.4          | 0.2         | Iris-setosa |
| 1 | 4.9          | 3.0         | 1.4          | 0.2         | Iris-setosa |
| 2 | 4.7          | 3.2         | 1.3          | 0.2         | Iris-setosa |
| 3 | 4.6          | 3.1         | 1.5          | 0.2         | Iris-setosa |
| 4 | 5.0          | 3.6         | 1.4          | 0.2         | Iris-setosa |
| 5 | 5.4          | 3.9         | 1.7          | 0.4         | Iris-setosa |
| 6 | 4.6          | 3.4         | 1.4          | 0.3         | Iris-setosa |
| 7 | 5.0          | 3.4         | 1.5          | 0.2         | Iris-setosa |
| 8 | 4.4          | 2.9         | 1.4          | 0.2         | Iris-setosa |
| 9 | 4.9          | 3.1         | 1.5          | 0.1         | Iris-setosa |

# Data preprocessing

First, we need to separate the features from the classes

- Different preprocessing for features and classes (Necessary? YES! To avoid Data Leakage!!!)

```
[77] y = dataset['iris_class']  
     x = dataset.drop(['iris_class'], axis=1)  
  
print ("dataset : ",dataset.shape)  
print ("x : ",x.shape)  
print ("y : ",y.shape)
```

Whether to drop  
labels from the  
index (0 or 'index')  
or columns  
(1 or 'columns').

As size() in Matlab

# Data preprocessing

First, we need to separate the features from the classes

- Feature matrix has 4 columns (the features)
- Target vector has 1 column (the class)

```
↳ dataset : (150, 5)  
   x : (150, 4)  
   y : (150, )
```

# Data preprocessing

Use the numpy library to convert data into matrix format

- Ease of access to portions of the matrices

```
[78] xNumpy = x.to_numpy()  
     yNumpy = y.to_numpy()
```

# Data preprocessing

We can access a single feature or a single samples

- Read a specific feature: matrix slicing by **column**

```
[79] feature_to_extract = 1  
     feature = xNumpy[:,feature_to_extract]  
     print(feature)
```

# Data preprocessing

Reading a specific feature: matrix slicing by **column**

```
[79] feature_to_extract = 1  
     feature = xNumpy[:,feature_to_extract]  
     print(feature)
```

```
☞ [3.5 3.  3.2 3.1 3.6 3.9 3.4 3.4 2.9 3.1 3.7 3.4 3.  3.  4.  4.4 3.9 3.5  
   3.8 3.8 3.4 3.7 3.6 3.3 3.4 3.  3.4 3.5 3.4 3.2 3.1 3.4 4.1 4.2 3.1 3.2  
   3.5 3.1 3.  3.4 3.5 2.3 3.2 3.5 3.8 3.  3.8 3.2 3.7 3.3 3.2 3.2 3.1 2.3  
   2.8 2.8 3.3 2.4 2.9 2.7 2.  3.  2.2 2.9 2.9 3.1 3.  2.7 2.2 2.5 3.2 2.8  
   2.5 2.8 2.9 3.  2.8 3.  2.9 2.6 2.4 2.4 2.7 2.7 3.  3.4 3.1 2.3 3.  2.5  
   2.6 3.  2.6 2.3 2.7 3.  2.9 2.9 2.5 2.8 3.3 2.7 3.  2.9 3.  3.  2.5 2.9  
   2.5 3.6 3.2 2.7 3.  2.5 2.8 3.2 3.  3.8 2.6 2.2 3.2 2.8 2.8 2.7 3.3 3.2  
   2.8 3.  2.8 3.  2.8 3.8 2.8 2.8 2.6 3.  3.4 3.1 3.  3.1 3.1 3.1 2.7 3.2  
   3.3 3.  2.5 3.  3.4 3. ]
```

# Data preprocessing

We can access  
a single feature  
or a single samples

- Read a specific feature: matrix slicing by column

```
[79] feature_to_extract = 1
      feature = xNumpy[:,feature_to_extract]
      print(feature)
```

## Debugging!!

```
[3.5 3. 3.2 3.1 3.6 3.9 3.4 3.4 2.9 3.1 3.7 3.4 3. 3. 4. 4.4 3.9 3.5
 2.8 2.8 2.4 2.7 2.6 2.2 2.4 2. 2.4 2.5 2.4 2.2 2.1 2.4 4.1 4.2 3.1 3.2
 3.5 3.1 3. 3.4 3.5 2.3 3.2 3.5 3.8 3. 3.8 3.2 3.7 3.3 3.2 3.2 3.1 2.3
 2.8 2.8 3.3 2.4 2.9 2.7 2. 3. 2.2 2.9 2.9 3.1 3. 2.7 2.2 2.5 3.2 2.8
 2.5 2.8 2.9 3. 2.8 3. 2.9 2.6 2.4 2.4 2.7 2.7 3. 3.4 3.1 2.3 3. 2.5
 2.6 3. 2.6 2.3 2.7 3. 2.9 2.9 2.5 2.8 3.3 2.7 3. 2.9 3. 3. 2.5 2.9
 2.5 3.6 3.2 2.7 3. 2.5 2.8 3.2 3. 3.8 2.6 2.2 3.2 2.8 2.8 2.7 3.3 3.2
 2.8 3. 2.8 3. 2.8 3.8 2.8 2.8 2.6 3. 3.4 3.1 3. 3.1 3.1 3.1 2.7 3.2
 3.3 3. 2.5 3. 3.4 3. ]
```

|   | sepal_length | sepal_width | petal_length | petal_width | iris_class  |
|---|--------------|-------------|--------------|-------------|-------------|
| 0 | 5.1          | 3.5         | 1.4          | 0.2         | Iris-setosa |
| 1 | 4.9          | 3.0         | 1.4          | 0.2         | Iris-setosa |
| 2 | 4.7          | 3.2         | 1.3          | 0.2         | Iris-setosa |
| 3 | 4.6          | 3.1         | 1.5          | 0.2         | Iris-setosa |
| 4 | 5.0          | 3.6         | 1.4          | 0.2         | Iris-setosa |
| 5 | 5.4          | 3.9         | 1.7          | 0.4         | Iris-setosa |
| 6 | 4.6          | 3.4         | 1.4          | 0.3         | Iris-setosa |
| 7 | 5.0          | 3.4         | 1.5          | 0.2         | Iris-setosa |
| 8 | 4.4          | 2.9         | 1.4          | 0.2         | Iris-setosa |
| 9 | 4.9          | 3.1         | 1.5          | 0.1         | Iris-setosa |

|   | sepal_length | sepal_width | petal_length | petal_width | iris_class  |
|---|--------------|-------------|--------------|-------------|-------------|
| 0 | 5.1          | 3.5         | 1.4          | 0.2         | Iris-setosa |
| 1 | 4.9          | 3.0         | 1.4          | 0.2         | Iris-setosa |
| 2 | 4.7          | 3.2         | 1.3          | 0.2         | Iris-setosa |
| 3 | 4.6          | 3.1         | 1.5          | 0.2         | Iris-setosa |
| 4 | 5.0          | 3.6         | 1.4          | 0.2         | Iris-setosa |
| 5 | 5.4          | 3.9         | 1.7          | 0.4         | Iris-setosa |
| 6 | 4.6          | 3.4         | 1.4          | 0.3         | Iris-setosa |
| 7 | 5.0          | 3.4         | 1.5          | 0.2         | Iris-setosa |
| 8 | 4.4          | 2.9         | 1.4          | 0.2         | Iris-setosa |
| 9 | 4.9          | 3.1         | 1.5          | 0.1         | Iris-setosa |

# Data preprocessing

We can access a single feature or a single samples

- Read a specific sample: matrix slicing by **row**

```
[80] sample_to_extract = 1
      sample = xNumpy[sample_to_extract,:]
      print(sample)
```

# Data preprocessing

We can access a single feature or a single samples

- Read a specific sample: matrix slicing by row

```
[80] sample_to_extract = 1  
     sample = xNumpy[sample_to_extract,:]   
     print(sample)
```

Debugging!!

➡ [4.9 3. 1.4 0.2]

|   | sepal_length | sepal_width | petal_length | petal_width | iris_class  |
|---|--------------|-------------|--------------|-------------|-------------|
| 0 | 5.1          | 3.5         | 1.4          | 0.2         | Iris-setosa |
| 1 | 4.9          | 3.0         | 1.4          | 0.2         | Iris-setosa |
| 2 | 4.7          | 3.2         | 1.3          | 0.2         | Iris-setosa |
| 3 | 4.6          | 3.1         | 1.5          | 0.2         | Iris-setosa |
| 4 | 5.0          | 3.6         | 1.4          | 0.2         | Iris-setosa |
| 5 | 5.4          | 3.9         | 1.7          | 0.4         | Iris-setosa |
| 6 | 4.6          | 3.4         | 1.4          | 0.3         | Iris-setosa |
| 7 | 5.0          | 3.4         | 1.5          | 0.2         | Iris-setosa |
| 8 | 4.4          | 2.9         | 1.4          | 0.2         | Iris-setosa |
| 9 | 4.9          | 3.1         | 1.5          | 0.1         | Iris-setosa |

# Data preprocessing

We can access a  
single feature  
or a single samples

- Read a specific sample: matrix slicing by row

|   | sepal_length | sepal_width | petal_length | petal_width | iris_class  |
|---|--------------|-------------|--------------|-------------|-------------|
| 0 | 5.1          | 3.5         | 1.4          | 0.2         | Iris-setosa |
| 1 | 4.9          | 3.0         | 1.4          | 0.2         | Iris-setosa |
| 2 | 5.0          | 3.2         | 1.5          | 0.2         | Iris-setosa |
| 3 | 4.6          | 3.1         | 1.5          | 0.2         | Iris-setosa |
| 4 | 5.0          | 3.6         | 1.4          | 0.2         | Iris-setosa |
| 5 | 5.4          | 3.9         | 1.7          | 0.4         | Iris-setosa |
| 6 | 4.6          | 3.4         | 1.4          | 0.3         | Iris-setosa |
| 7 | 5.0          | 3.4         | 1.5          | 0.2         | Iris-setosa |
| 8 | 4.4          | 2.9         | 1.4          | 0.2         | Iris-setosa |
| 9 | 4.9          | 3.1         | 1.5          | 0.1         | Iris-setosa |

```
[80] sample_to_extract = 1
      sample = xNumpy[sample_to_extract,:]
      print(sample)
```

```
☞ [4.9 3.  1.4 0.2]
```

# Data preprocessing

Data normalization:

ensure that the data is in the range [0-1]

- Min-max normalization:  
$$(x - \min(x)) / (\max(x) - \min(x))$$

```
[81] xNumpy_Norm = (xNumpy - np.min(xNumpy)) / (np.max(xNumpy) - np.min(xNumpy))
```

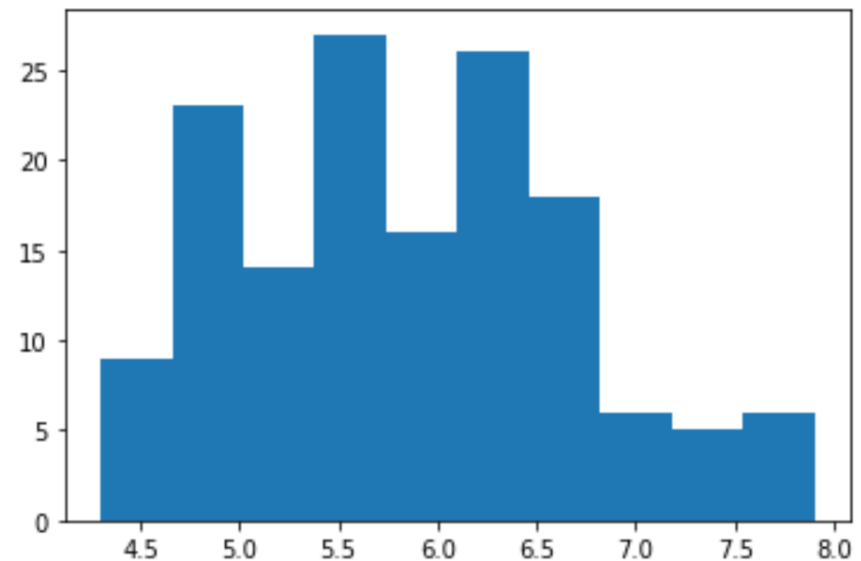
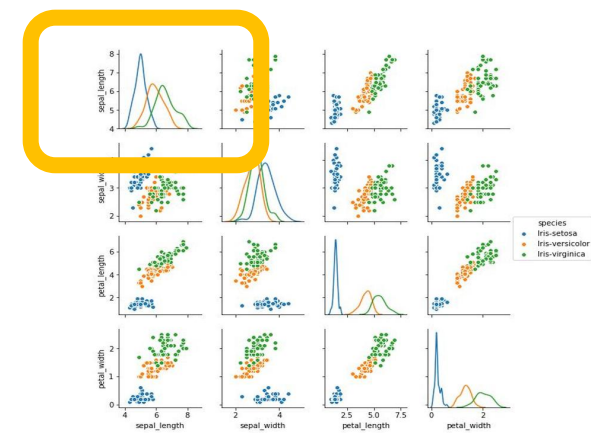
# Data preprocessing

Data normalization:  
ensure that the data is in the range [0-1]

- Non-normalized data

```
plt.subplots()  
plt.hist(xNumpy[:,0])
```

|   | sepal_length | sepal_width | petal_length | petal_width | iris_class  |
|---|--------------|-------------|--------------|-------------|-------------|
| 0 | 5.1          | 3.5         | 1.4          | 0.2         | Iris-setosa |
| 1 | 4.9          | 3.0         | 1.4          | 0.2         | Iris-setosa |
| 2 | 4.7          | 3.2         | 1.3          | 0.2         | Iris-setosa |
| 3 | 4.6          | 3.1         | 1.5          | 0.2         | Iris-setosa |
| 4 | 5.0          | 3.6         | 1.4          | 0.2         | Iris-setosa |
| 5 | 5.4          | 3.9         | 1.7          | 0.4         | Iris-setosa |
| 6 | 4.6          | 3.4         | 1.4          | 0.3         | Iris-setosa |
| 7 | 5.0          | 3.4         | 1.5          | 0.2         | Iris-setosa |
| 8 | 4.4          | 2.9         | 1.4          | 0.2         | Iris-setosa |
| 9 | 4.9          | 3.1         | 1.5          | 0.1         | Iris-setosa |



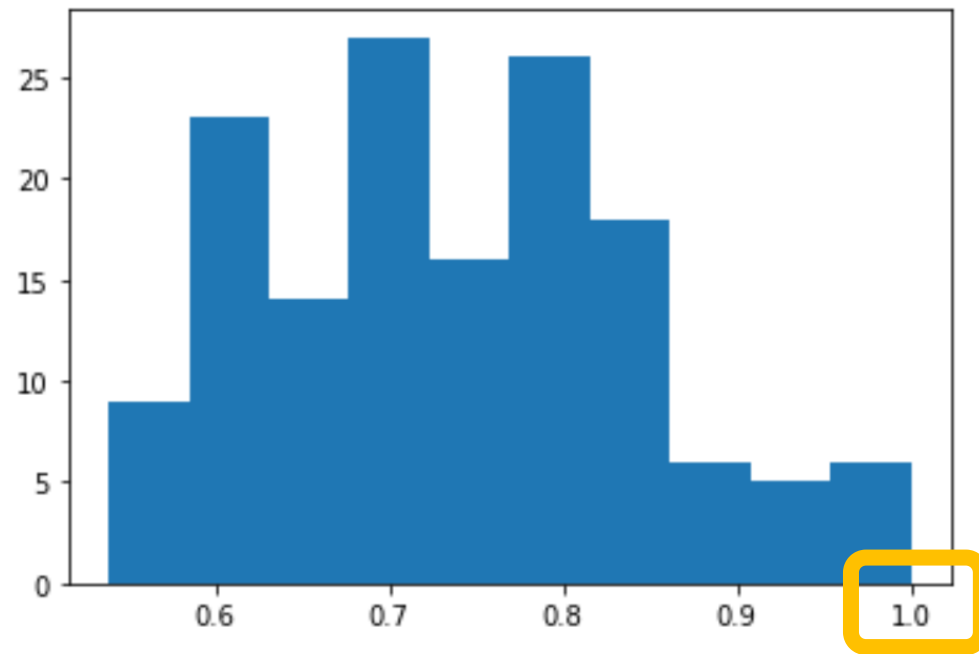
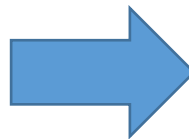
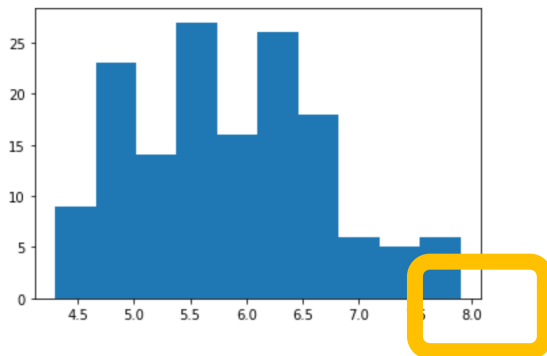
```
[81] xNumpy_Norm = (xNumpy - np.min(xNumpy)) / (np.max(xNumpy) - np.min(xNumpy))
```

# Data preprocessing

Data normalization:  
ensure that the data is in the range [0-1]

- Normalized data

```
plt.subplots()  
plt.hist(xNumpy_Norm[:,0])
```



# Data preprocessing

Data conversion: from categorical to decimal

- Class data is in categorical format

**iris\_class**

Iris-setosa

Iris-setosa

Iris-setosa

Iris-setosa

Iris-setosa

Iris-setosa

Iris-setosa

Iris-setosa

Iris-setosa

Iris-setosa

# Data preprocessing

# Data conversion: from categorical to decimal

- Conversion to decimal for easier manipulation

```
▶ yNumpy, dummy = pd.factorize(yNumpy)
print(yNumpy)
```

Diagram illustrating a sliding window of size 10 over a sequence of 25 elements. The window is currently positioned over the first 10 elements, which are all 0s. A blue arrow indicates the window moving to the right, covering the next 10 elements (9 0s and 1 1). The remaining 5 elements (1 1s and 3 2s) are outside the window.

iris\_class

Iris-setosa

Iris-setosa

Iris-setosa

Iris-setosa

Iris-setosa

Iris-rosa

Iris-rosa

Iris-rosa

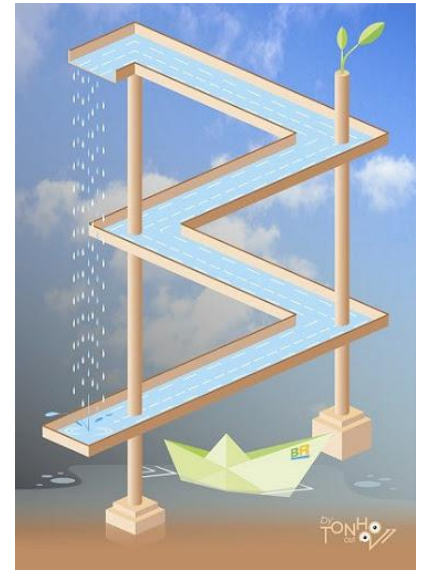
Iris

Iris-setosa

# Data visualization

Visualizing the features and the classes allows us to have a first visual check on the discriminatory capability

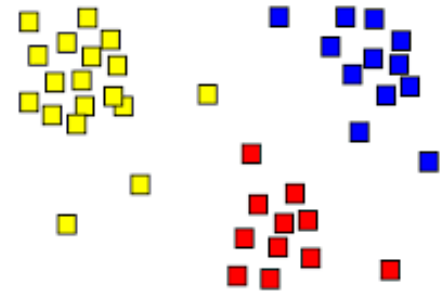
- Drawback: visualization is limited to 2 or 3 dimensions
- Even with a simple dataset like the IRIS dataset (4D), it's impossible to plot all features with their axis.



# Using Data visualization to manage the 4 dims of IRIS

Visualizing the features and the classes allows us to have a first **visual check on the discriminatory capability**

1. Select features to plot
2. Assign color based on the class

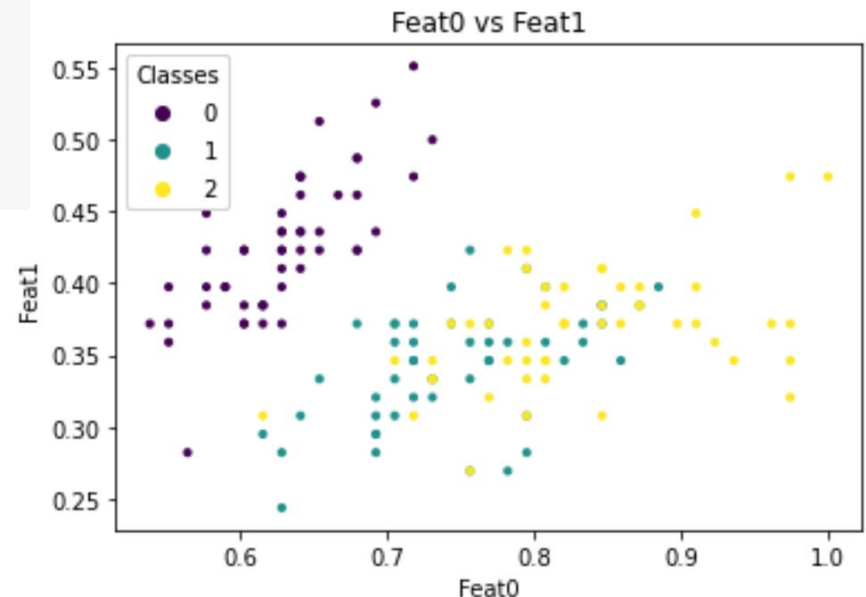


# Data visualization

```
import matplotlib.pyplot as plt
# select 2 features to plot
feat0 = xNumpy_Norm[:,0]
feat1 = xNumpy_Norm[:,1]
# params
area = np.pi*3
# Plot
fig1, ax1 = plt.subplots()
scatter1 = ax1.scatter(feat0, feat1, s=area, c=yNumpy)
plt.title('Feat0 vs Feat1')
plt.xlabel('Feat0')
plt.ylabel('Feat1')
legend1 = ax1.legend(*scatter1.legend_elements(),
                    loc="upper left", title="Classes")
ax1.add_artist(legend1)
plt.show()
```

Feat0 + Feat1:

- Good discrimination for class 0
- Bad discrimination for class 1 and 2 (classes are mixed)

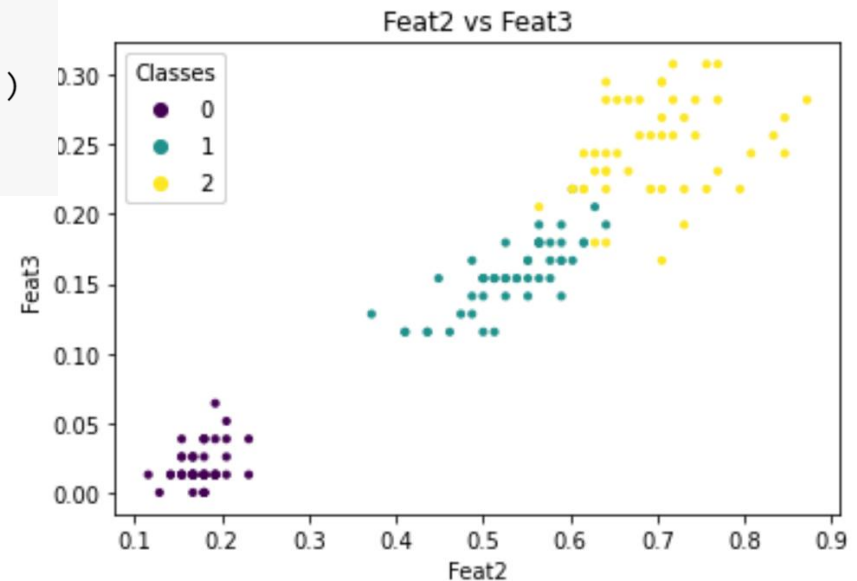


# Data visualization

```
# select 2 features to plot
feat2 = xNumpy_Norm[:,2]
feat3 = xNumpy_Norm[:,3]
# params
area = np.pi*3
# Plot
fig2, ax2 = plt.subplots()
scatter2 = ax2.scatter(feat2, feat3, s=area, c=yNumpy)
plt.title('Feat2 vs Feat3')
plt.xlabel('Feat2')
plt.ylabel('Feat3')
legend2 = ax2.legend(*scatter2.legend_elements(),
                    loc="upper left", title="Classes")
ax2.add_artist(legend2)
plt.show()
```

Feat2 + Feat3:

- Better discrimination for all classes



# Main points

- Similarity as a tool for deep learning
- Exploratory Data Analysis
- Feature engineering
  - Data preprocessing:  
a statistical point of view
- Application of EDA to NN
  - Inputs
  - Weights
  - Activations
  - Outputs

